

THE UNIVERSITY OF ILLINOIS AT CHICAGO

Department of Chemical Engineering

MIDTERM #2

Course: CHE 341. Chemical Process Control

Instructor: Dr. Michael Caracotsios

E-mail: mcaracot@uic.edu

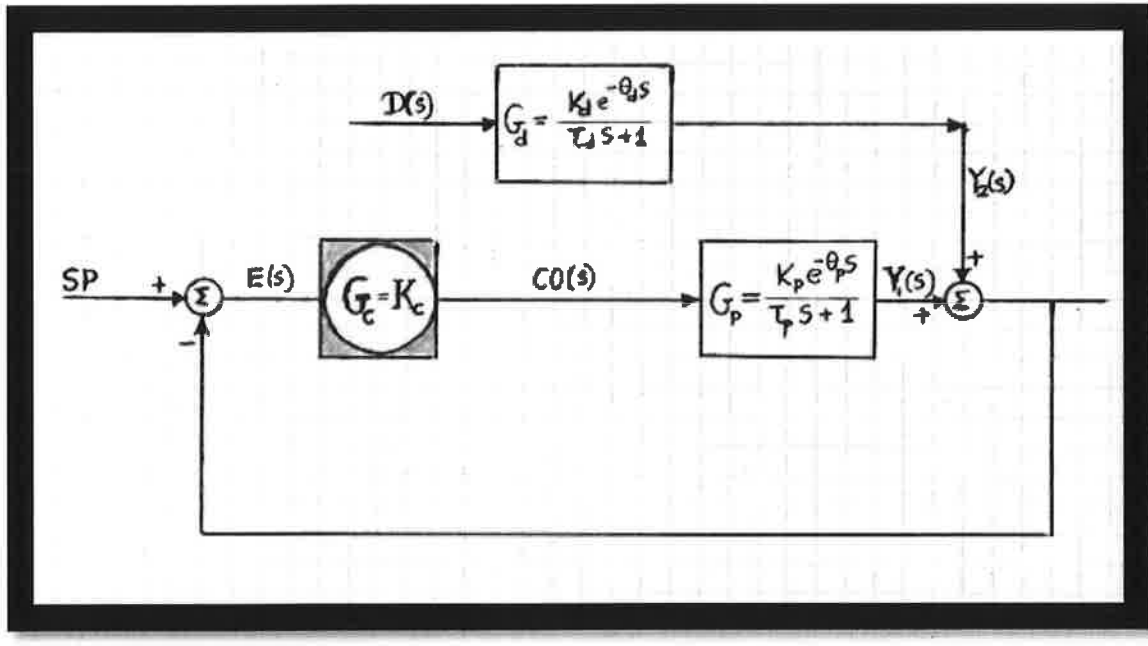
Your name: Shereen Daik

Total number of points..... 60

Your score.....

Problem 1:

- I. **Problem Statement:** Devise two different ways to derive the controller gain functions. Calculate the true controller gain that corresponds to critical damping using the modeling tool for a set point step change equal to $SP=10$ using a trial and error process. Calculate the error in your estimation via the two different methods.



II. Given:

τ_p	25.0
θ_p	8.47
K_p	6.73
G_p	$\frac{K_p e^{-\theta_p s}}{\tau_p s + 1}$
G_c	K_c

III. Derivations :

Derivation #1: Using Pade's 1/1 Approximation

Begin with developing a closed loop transfer function for the system:

$$Y(s) = G_d D(s) + G_p G_c (SP - Y(s)) \quad (1)$$

$$Y(s) = G_d D(s) + G_p G_c SP - G_p G_c Y(s) \quad (2)$$

$$Y(s) + G_P G_C Y(s) = G_d D(s) + G_P G_C SP \quad (3)$$

$$Y(s)(1 + G_P G_C) = G_d D(s) + G_P G_C SP \quad (4)$$

$$Y(s) = \frac{G_d D(s) + G_P G_C SP}{1 + G_P G_C} \quad (5)$$

The characteristic equation is then equivalent to the denominator of the transfer function:

$$1 + G_P G_C = 0 \quad (6)$$

$$\text{It is given that } G_P = \frac{K_P e^{-\theta_P s}}{\tau_P s + 1} \text{ and } G_C = K_C \quad (7)$$

Thus, the characteristic equation becomes:

$$1 + \frac{K_P e^{-\theta_P s}}{\tau_P s + 1} * K_C = 0 \quad (8)$$

Pade's approximation states that $e^{-\theta s} = \frac{1 - \frac{\theta s}{2}}{1 + \frac{\theta s}{2}}$, therefore $e^{-\theta_P s} = \frac{1 - \frac{\theta_P s}{2}}{1 + \frac{\theta_P s}{2}}$

$$1 + \frac{K_P (1 - \frac{\theta_P s}{2})}{(\tau_P s + 1)(1 + \frac{\theta_P s}{2})} * K_C = 0 \quad (9)$$

$$(\tau_P s + 1)(1 + \frac{\theta_P s}{2}) + K_P (1 - \frac{\theta_P s}{2}) * K_C = 0$$

Simplifying gives the following characteristic equation:

$$s^2(\tau_P \theta) + s(2\tau_P + \theta - K_P K_C \theta) + 2(1 + K_P K_C) = 0 \quad (10)$$

The above equation (10) greatly resembles the denominator of the second order transfer function shown below:

$$Y(s) = \frac{K_P e^{-\theta s}}{\tau^2 s^2 + 2\tau\zeta s + 1} \quad (11)$$

Let:

$$\frac{\tau_P \theta}{2(1+K_P K_C)} = \tau^2 \quad (12)$$

and

$$\frac{(2\tau_P + \theta - K_P K_C \theta)}{2(1+K_P K_C)} = 2\tau\zeta \quad (13)$$

For a critically damped system $\zeta=1$, therefore:

$$\tau = \frac{(2\tau_P + \theta - K_P K_C \theta)}{4(1+K_P K_C)} \quad (14)$$

By combining equations (13) and (14) we find that...

$$\left[\frac{(2\tau_P + \theta - K_P K_C \theta)}{4(1+K_P K_C)} \right]^2 = \frac{\tau_P \theta}{2(1+K_P K_C)} \quad (15)$$

Rearranging and simplifying equation (15) gives:

$$(2\tau_P + \theta - K_C K_P)^2 = 8\tau_P \theta_P (1 + K_C K_P) \quad (16)$$

Create an equation for K_C

$$K_C = \frac{2\sqrt{2} \sqrt{K_P^2 \tau_P (2\tau_P + 1) \theta_P (\theta_P + 1) + K_P (\tau_P (4\theta_P + 2) + \theta_P)}}{K_P^2} \quad (17)$$

Plug in $\tau_P=25$, $\theta_P=8.47$, and $K_P=6.73$ to solve for K_C

$$K_C=0.0972$$

Calculate the percent error between the controller gain found above and the true controller gain (0.133) using equation (18) below.

$$\% \text{ Error} = \left| \frac{\text{Theoretical Value} - \text{Experimental Value}}{\text{Theoretical Value}} \right| * 100 \quad (18)$$

$$\% \text{ Error} = \left| \frac{0.133 - 0.0972}{0.133} \right| * 100 = 26.92 \%$$

Derivation #2: Using Taylor's Expansion

Once again begin by devising the transfer function for the system (Refer to the derivation leading to eq. (2))

$$Y(s) = \frac{G_D D(s) + G_P G_C SP}{1 + G_P G_C} \quad (2A)$$

For this derivation set $D(s)=0$, Equation 2A then becomes:

$$\frac{Y(s)}{SP} = \frac{G_C G_P}{1 + G_C G_P} \quad (3A)$$

Therefore, the close loop transfer function is equal to the denominator of equation 3A:

$$1 + G_C G_P = 0 \quad (4A)$$

It is given that $G_P = \frac{K_P e^{-\theta_P s}}{\tau_P s + 1}$ and $G_C = K_C$

$$1 + K_C \frac{K_P e^{-\theta_P s}}{\tau_P s + 1} = 0 \quad (5A)$$

Using Taylor Series Expansion $e^{-\theta_P s} = 1 - \theta_P s + \frac{(\theta_P s)^2}{2}$, substitute this into equation 5A to get...

$$1 + K_C \frac{K_P}{(\tau_P s + 1)} * (1 - \theta_P s + \frac{(\theta_P s)^2}{2}) = 0 \quad (6A)$$

Rearrange equation 6A to arrive at the characteristic equation (7A):

$$2(\tau_P s + 1) + 2K_C K_P - 2K_C K_P \theta_P s - K_C K_P (\theta_P s)^2 = 0 \quad (7A)$$

$$(1 + K_C K_P) + (\tau_P - K_C K_P \theta_P) s - \frac{s^2}{2} (K_C K_P \theta_P^2) = 0 \quad (8A)$$

The above equation (8A) greatly resembles the denominator of the second order transfer function shown below:

$$Y(s) = \frac{K_P e^{-\theta s}}{\tau^2 s^2 + 2\tau\zeta s + 1}$$

(9A)

Let:

$$\frac{(K_C K_P \theta_P^2)}{2(1 + K_C K_P)} = \tau^2 \quad (10A)$$

and

$$\frac{(\tau_p - K_C K_P \theta_p)}{(1 + K_P K_C)} = 2\tau\zeta \quad (11A)$$

For a critically damped system $\zeta=1$, therefore:

$$\tau = \frac{(\tau_p - K_C K_P \theta_p)}{2(1 + K_P K_C)} \quad (12A)$$

By combining equations (11A) and (12A) we find that...

$$\left[\frac{(\tau_p - K_C K_P \theta_p)}{2(1 + K_P K_C)} \right]^2 = \frac{(K_C K_P \theta_p^2)}{2(1 + K_C K_P)} \quad (13A)$$

Rearranging and simplifying equation (3A) gives:

$$(\tau_p - K_C K_P \theta_p)^2 = 2(K_C K_P \theta_p^2)(1 + K_C K_P) \quad (14A)$$

Rewrite to form an equation for K_C :

$$K_C = \frac{\sqrt{\theta_p^2 K_P^2 (\theta_p^2 + 2\tau_p \theta_p + 2\tau_p^2) + K_P \theta_p^2 + \theta_p \tau_p K_P}}{K_P^2 \theta_p} \quad (15A)$$

Plug in $\tau_p=25$, $\theta_p=8.47$, and $K_P=6.73$ to solve for K_C

$$K_C=0.146$$

Calculate the percent error between the controller gain found above and the true controller gain (0.133) using equation (16A) below.

$$\% \text{ Error} = \left| \frac{\text{Theoretical Value} - \text{Experimental Value}}{\text{Theoretical Value}} \right| * 100 \quad (16A)$$

$$\% \text{ Error} = \left| \frac{0.133 - 0.146}{0.133} \right| * 100 = 9.7 \%$$

Derivation #3: Using Reverse Skogestad

Once again begin by devising the transfer function for the system (Refer to the derivation leading to eq. (2))

$$Y(s) = \frac{G_D D(s) + G_P G_C S P}{1 + G_P G_C} \quad (2B)$$

Therefore, the close loop transfer function is equal to the denominator of equation 3A:

$$1 + G_C G_P = 0 \quad (3B)$$

By plugging in the given values for G_C and G_P to simplify equation (3B)

$$1 + K_c * \frac{K_p e^{-\theta_p s}}{\tau_p s + 1} \quad (4B)$$

For the standard form of the second order transfer function:

$$\frac{K_p}{(\tau_1 s + 1)(\tau_2 s + 1)} \quad (5B)$$

Make the assumption that $\tau_1 > \tau_2$ to get:

$$\tau_p = \tau_1 + \frac{\tau_2}{2} \quad (6B)$$

$$\theta_p = \frac{\tau_2}{2} \quad (7B)$$

Plug equation 7B into 6B,

$$\tau_1 = \tau_p - \theta_p \quad (8B)$$

Plug equation 8B into equation 5B:

$$\frac{K_p}{((\tau_p - \theta_p)s + 1)(2\theta_p s + 1)} \quad (9B)$$

Simplify equation 9B to and plug into 4B to get:

$$1 + \frac{K_c K_p}{(2\theta_p \tau_p - 2\theta_p)s^2 + (\tau_p + \theta_p)s + 1} = 0 \quad (10B)$$

Therefore, the characteristic equation becomes:

$$(2\theta_p\tau_p - 2\theta_p)s^2 + (\tau_p + \theta_p)s + (1 + K_c K_p) \quad (11B)$$

The above equation (11B) greatly resembles the denominator of the second order transfer function shown below:

$$Y(s) = \frac{K_p e^{-\theta s}}{\tau^2 s^2 + 2\tau\zeta s + 1} \quad (12B)$$

Let:

$$\frac{2\theta_p\tau_p - 2\theta_p}{(1 + K_c K_p)} = \tau^2 \quad (13B)$$

and

$$\frac{(\tau_p + \theta_p)}{(1 + K_p K_c)} = 2\tau\zeta \quad (14B)$$

For a critically damped system $\zeta=1$, therefore:

$$\tau = \frac{(\tau_p - \theta_p)}{2(1 + K_p K_c)} \quad (15B)$$

By combining equations (14B) and (15B) we find that...

$$\left[\frac{(\tau_p - \theta_p)}{2(1 + K_p K_c)} \right]^2 = \frac{2\theta_p\tau_p - 2\theta_p}{(1 + K_c K_p)} \quad (16B)$$

Rearranging and simplifying equation (3A) gives:

$$(\tau_p - \theta_p)^2 = 4(2\tau_p\theta_p - 2\theta_p^2)(1 + K_c K_p)^2 \quad (17B)$$

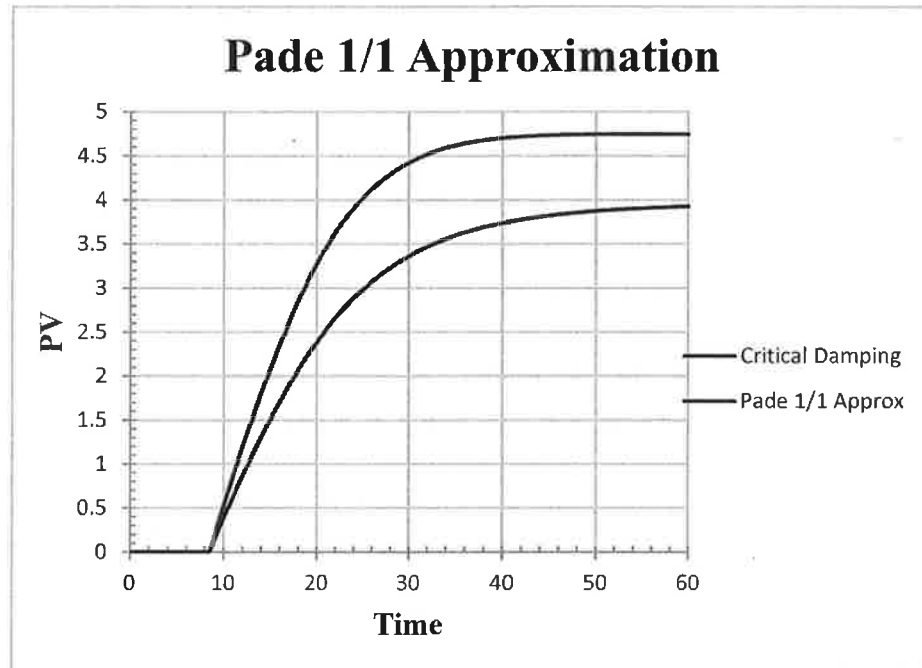
Plug in given values: $\tau_p = 25$, $\theta_p = 8.47$, $K_p = 6.73$ and then use goal seek to solve for K_c .

$$K_c = 2.23 \times 10^{-5}$$

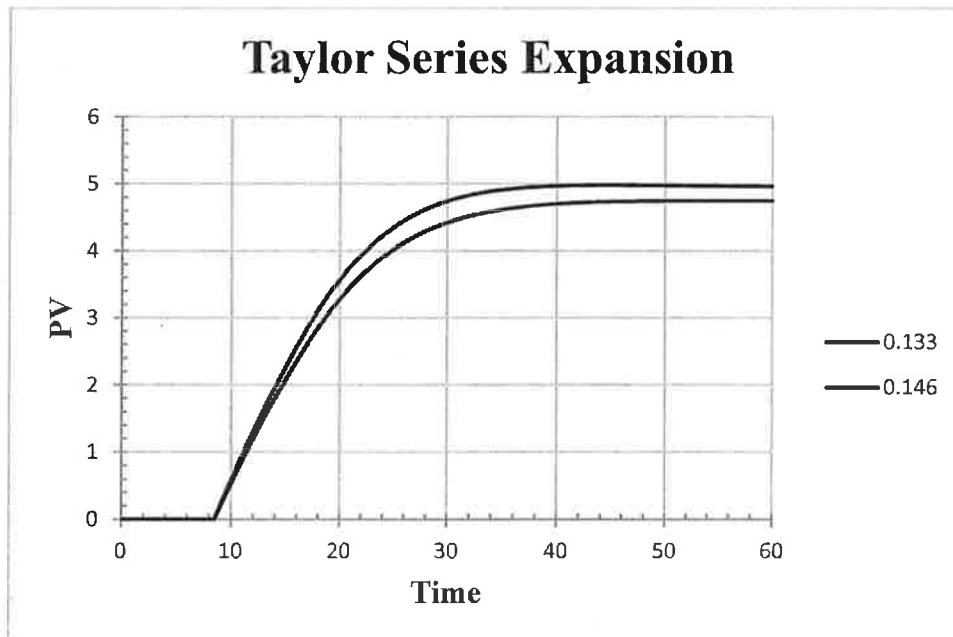
This value is non-comparable to the true gain value.

IV. Plots

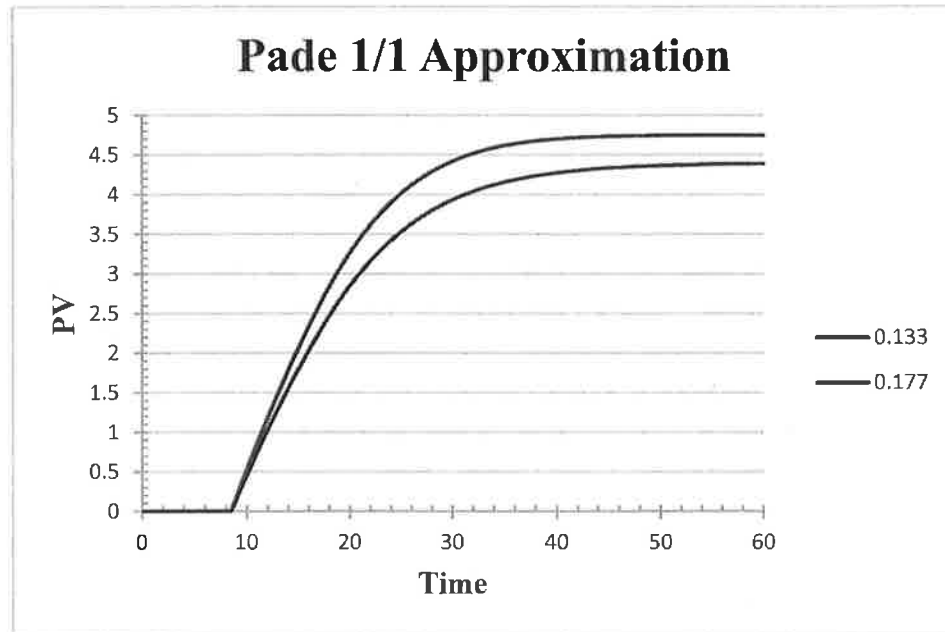
- A. Process Response for a set point step change equal to $SP=10$ with the true gain corresponding to critical damping vs. response using gain from first method (Pade's 1/1 Approximation)



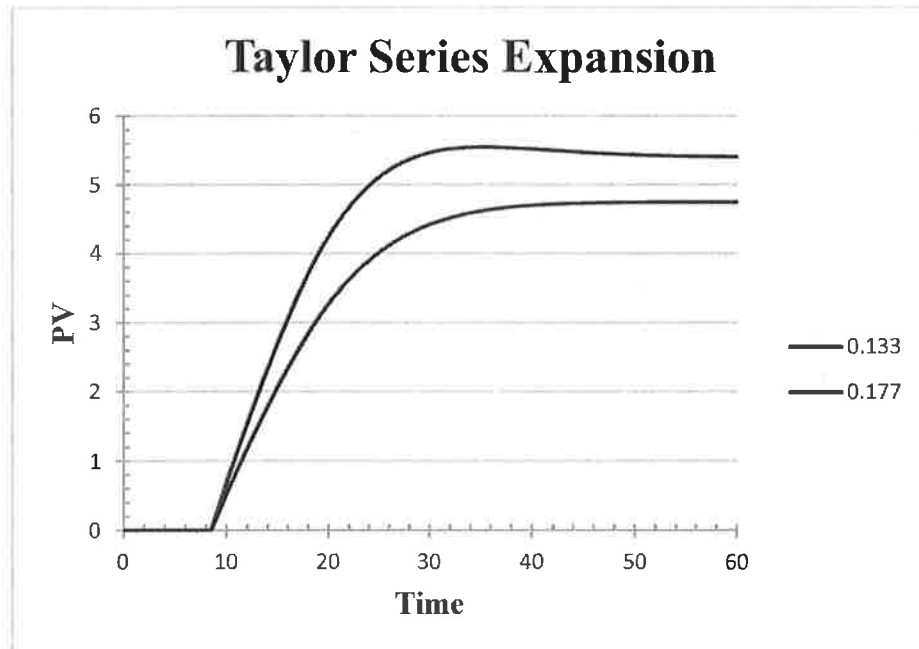
- B. Process response for a set point step change equal to $SP=10$ with true gain corresponding to critical damping vs. response using gain from second method (Taylor Series Expansion)



- C. Process response for a set point step change equal to $SP=10$ with the gain you estimated from first method (0.0972), but increased to a new level +20% higher (0.117)



- D. Process response for a set point step change equal to $ISP=10$ with the gain you estimated from the second method (0.146), but increased to a new level +20% higher (0.177)



Discussion

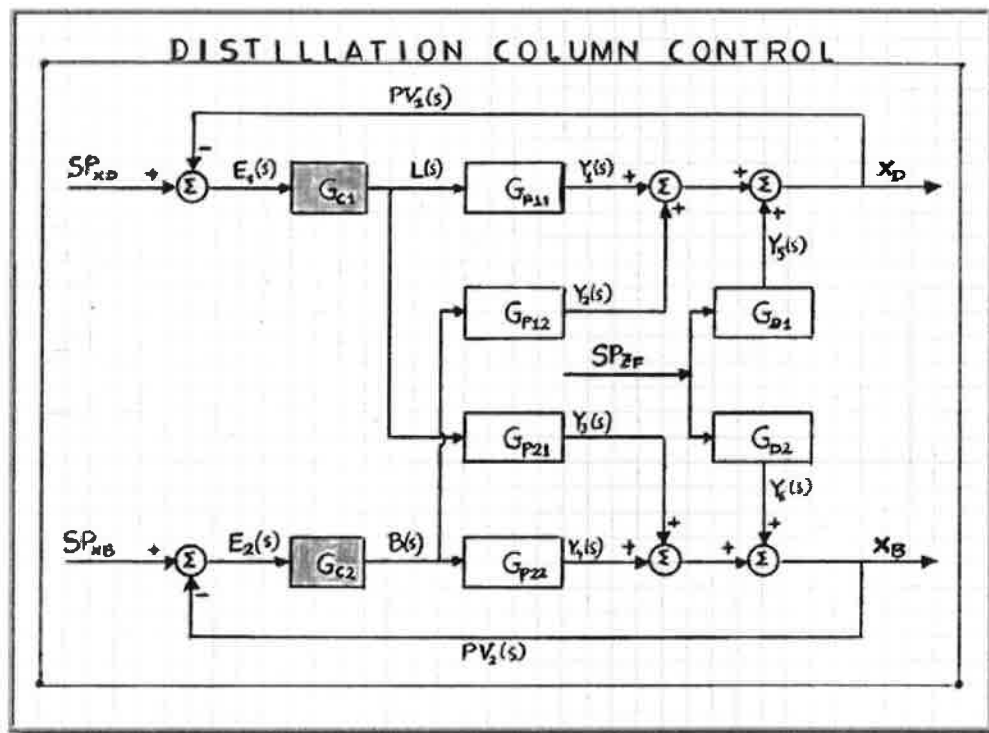
It appears as though the plot for Part C is critically damped and Part D is underdamped.

Problem 2:

I. Problem Statement:

(a) Derive the differential equation for $Y_1(t)$, $Y_2(t)$, $Y_3(t)$, $Y_4(t)$, $PV_1(t)$, $PV_2(t)$ as a function of $L(t)$ and $B(t)$.

(b) If the two proposed controllers $GC1$ and $GC2$ are proportional-integral with controller gains $KC1=32.08$ and $KC2=3.58$, derive the ranges of their integral time constants τ_{I1} and τ_{I2} for stable performance using Routh criterion.



(a):

→ $Y_1(t)$:

$$Y_1(s) = L(s)G_{P11} \quad (2.1A)$$

$$Y_1(s) = L(s) \left[\frac{0.1247}{10s + 1} \right] \quad (2.1B)$$

$$10 * s * Y_1(s) + Y_1(s) = 0.1247 * L(s) \quad (2.1C)$$

$$10 \frac{dY_1(t)}{dt} + Y_1(t) = 0.1247 * L(t) \quad (2.1D)$$

→ $Y_2(t)$:

$$Y_2(s) = B(s)G_{P12} \quad (2.2A)$$

$$Y_2(s) = B(s) \left[\frac{0.546}{24s + 1} \right] \quad (2.2B)$$

$$24 * s * Y_2(s) + Y_2(s) = 0.546 * B(s) \quad (2.2 C)$$

$$24 \frac{dY_2(t)}{dt} + Y_2(t) = 0.546 * B(t) \quad (2.2 D)$$

→ $Y_3(t)$:

$$Y_3(s) = L(s)G_{P21} \quad (2.3A)$$

$$Y_3(s) = L(s) \left[\frac{-0.1246}{16s + 1} \right] \quad (2.3B)$$

$$16 * s * Y_3(s) + Y_3(s) = -0.1246 * L(s) \quad (2.3 C)$$

$$16 \frac{dY_3(t)}{dt} + Y_3(t) = -0.1246 * L(t) \quad (2.3 D)$$

→ $Y_4(t)$:

$$Y_4(s) = B(s)G_{P22} \quad (2.4A)$$

$$Y_4(s) = B(s) \left[\frac{1.116}{22s + 1} \right] \quad (2.4B)$$

$$22 * s * Y_4(s) + Y_4(s) = 1.116 * B(s) \quad (2.4 C)$$

$$22 \frac{dY_4(t)}{dt} + Y_4(t) = 1.116 * B(t) \quad (2.4 D)$$

→ $PV_1(t)$:

$$PV_1(s) = Y_1(s) + Y_2(s) + Y_5(s) \quad (2.5A)$$

$$PV_1(t) = Y_1(t) + Y_2(t) + Y_5(t) \quad (2.5B)$$

$$Y_5(s) = SP_{ZF}G_{D1} \quad (2.5C)$$

$$Y_5(s) = SP_{ZF} \left[\frac{0.588e^{-7s}}{22s + 1} \right] \quad (2.5D)$$

$$22 * s * Y_5(s) + Y_5(s) = 0.588e^{-7s} * SP_{ZF} \quad (2.5 E)$$

$$22 \frac{dY_5(t)}{dt} + Y_5(t) = 0.588 * SP_{ZF} * (t - 7) \quad (2.5 D)$$

$$PV_1 = 0.1247L(t) - \frac{10dY_1(t)}{dt} + 0.546B(t) - \frac{24dY_2(t)}{dt} + 0.588 * SP_{ZF}(t - 7) - \frac{22dY_5(t)}{dt}$$

→ $PV_2(t)$:

$$PV_2(s) = Y_3(s) + Y_4(s) + Y_6(s) \quad (2.6A)$$

$$PV_2(t) = Y_3(t) + Y_4(t) + Y_6(t) \quad (2.6B)$$

$$Y_6(s) = SP_{ZF} G_{D2} \quad (2.6C)$$

$$Y_6(s) = SP_{ZF} \left[\frac{1.41e^{-7s}}{24s + 1} \right] \quad (2.6D)$$

$$24 * s * Y_6(s) + Y_6(s) = 1.41e^{-7s} * SP_{ZF} \quad (2.6E)$$

$$24 \frac{dY_6(t)}{dt} + Y_6(t) = 1.41 * SP_{ZF} * (t - 7) \quad (2.6D)$$

$$PV_2 = 1.116B(t) - \frac{22dY_4(t)}{dt} - 0.1246L(t) - \frac{16dY_3(t)}{dt} + 1.41 * SP_{ZF}(t - 7) - \frac{24dY_6(t)}{dt}$$

(b):

→ Range for τ_{I1} :

Begin by developing the closed loop transfer function for PV_1 :

$$PV_1 = G_{D1}SP_{ZF} + G_{P12}B(s) + G_{P11}G_{C1}(SP_{XD} - PV_1) \quad (2.7A)$$

$$PV_1(1 + G_{P11}G_{C1}) = G_{D1}SP_{ZF} + G_{P12}B(s) + G_{P11}G_{C1}SP_{XD} \quad (2.7B)$$

$$PV_1 = \frac{G_{D1}SP_{ZF}}{(1 + G_{P11}G_{C1})} + \frac{G_{P12}B(s)}{(1 + G_{P11}G_{C1})} + \frac{G_{P11}G_{C1}SP_{XD}}{(1 + G_{P11}G_{C1})} \quad (2.7C)$$

$$\text{Therefore, the characteristic equation is } 1 + G_{P11}G_{C1} = 0 \quad (2.7D)$$

The problem statement gives that:

G_{C1}	$K_{c1} \left(1 + \frac{1}{\tau_{I1}} \right)$
G_{P11}	$\frac{0.1247}{10s + 1}$

Thus, equation (2.7 D) becomes:

$$1 + \frac{0.1247}{10s + 1} * 32.08 * \left(1 + \frac{1}{\tau_{I1}} \right) = 0 \quad (2.7E)$$

Which simplifies too...

$$10\tau_I s^2 + 5\tau_I s + 4 = 0 \quad (2.7F)$$

Routh's Criterion states that $a_2=10\tau_I$ and $a_1=5\tau_I$ and $a_0=4$

Routh's criterion states that a_2 , a_1 , and a_0 should all be greater than zero. Since all of these terms are constants it can be assumed that the range for τ_I is greater than zero.

(c).

→ Range for τ_{I2} :

Begin by developing the closed loop transfer function for PV_1 :

$$PV_2 = G_{D2}SP_{ZF} + G_{P21}L(s) + G_{P22}G_{C2}(SP_{XB} - PV_2) \quad (2.8A)$$

$$PV_2(1 - G_{P22}G_{C2}) = G_{D2}SP_{ZF} + G_{P21}B(s) + G_{P22}G_{C2}SP_{XB} \quad (2.8B)$$

$$PV_1 = \frac{G_{D2}SP_{ZF}}{(1 + G_{P22}G_{C2})} + \frac{G_{P22}B(s)}{(1 + G_{P22}G_{C2})} + \frac{G_{P22}G_{C2}SP_{XB}}{(1 + G_{P22}G_{C2})} \quad (2.8C)$$

$$\text{Therefore, the characteristic equation is } (1 + G_{P22}G_{C2}) = 0 \quad (2.8D)$$

The problem statement gives that:

G_{C2}	$K_{c2} \left(1 + \frac{1}{\tau_{I2}} \right)$
G_{P22}	$\frac{1.116}{22s + 1}$

Thus, equation (2.7 D) becomes:

$$1 + \frac{1.116}{22s + 1} * 3.58 * \left(1 + \frac{1}{\tau_{I2}} \right) = 0 \quad (2.7E)$$

Which simplifies too...

$$22\tau_I s^2 + (4.995\tau_{I2})s + 3.995 = 0 \quad (2.7F)$$

Routh's Criterion states that $a_2=22\tau_{I2}$ and $a_1=4.995\tau_{I2}$ and $a_0=3.995$

Routh's criterion states that a_2 , a_1 , and a_0 should all be greater than zero. Since all of these terms are constants it can be assumed that the range for τ_{I1} is greater than zero.

For question 10 we are asked to use the three tuning methods :

1) Ziegler-Nichols

$$K_C = \left(\frac{0.9}{\text{Stability} * K_P} \right) * \left(\frac{\tau}{t_d} \right)$$

$$\tau_I = 3.33 * t_d$$

$$K_{CI} = \frac{K_C}{\tau_1}$$

2) Ziegler-Nichols Ultimate Gain:

Ziegler-Nichols Ultimate Gain:

$$K_C = \left(\frac{0.45 * K_U}{\text{Stability}} \right)$$

$$\tau_I = 0.83 * P_U$$

$$K_{CI} = \frac{K_C}{\tau_1}$$

In order to calculate K_U and P_U we use the following equations:

$$\tau_P \omega + \tan(\theta_P \omega) = 0$$

$$K_U = \left(-\frac{1}{K_P \cos(\theta_P \omega)} \right)$$

$$P_U = \frac{2\pi}{\omega}$$

Calculate of K_{U1} and P_{U1} using the following equations:

$$\tau_{p11} \omega + \tan(\theta_p \omega) = 0$$

$$10\omega + \tan(5\omega) = 0$$

Use goal seek to solve for $\omega = 0.367$:

Once we solve for ω we can solve for K_{U1} :

$$K_{U1} = \frac{-1}{K_{p11} \cos(\theta_p \omega)} = \frac{-1}{0.1247 \cos(5 * 0.367)} = 30.53$$

Using ω we can also solve for P_{U1} :

$$P_{U1} = \frac{2\pi}{\omega} = \frac{2 * 3.14}{0.3673} = 30.53$$

Calculation of K_{U2} and P_{U2} :

$$\tau_{p22}\omega + \tan(\theta_p\omega) = 0$$

$$22\omega + \tan(5\omega) = 0$$

Once again, use goal seek to solve for $\omega = 0.341$:

Now use ω to solve for K_{U2} :

$$K_{U2} = \frac{-1}{K_{p22}\cos(\theta_p\omega)} = \frac{-1}{1.116\cos(5 * 0.3407)} = 6.78$$

Similarly use ω to solve for P_{U2} :

$$P_{U2} = \frac{2\pi}{\omega} = \frac{2 * 3.14}{0.3407} = 18.43$$

3) Cohen-Coon

$$K_c = \frac{0.9}{SM * K_p} * \left(\frac{\tau}{t_d} + 0.092\right)$$

$$\tau_I = 3.33 * t_d * \left(\frac{\tau + 0.092t_d}{\tau + 2.22t_d}\right)$$

$$K_{cl} = \frac{K_c}{\tau_I}$$

Problem 2; Question 8:

IPID Controllers

!Regulatory-Control Implementation in Time-Domain

!Written by: Shereen Daik

!*****

Use AthenaUtilities

Global KC1,KC2,Kci1,Kci2,Kcd,SP1,SP2,DS,KP11,KP12 As Real
Global KP21,KP22,KPD1,KPD2,Tau11,Tau21,Tau12 As Real
Global Tau22,TauD1,TauD2,CO1,CO2,PV1,PV2 As Real
Global Time(5000),Y1i(5000),Y2i(5000),Y3i(5000),Y4i(5000)As
Real
Global Y5i(5000),Y6i(5000),CO1i(5000),CO2i(5000),PV1i(5000)
As Real
Global PV2i(5000),Ei1(5000),Ei2(5000),Thetap,Thetad As Real
Global iCount As Integer

Thetap=0.0 !SD,Process Dead Time
Thetad=7.0 !SD,Disturbance Dead time
Tau11=10.0 !SD,Process Lag
Tau21=16.0 !SD,Process Lag
Tau12=24.0 !SD,Process Lag
Tau22=22.0 !SD,Process Lag
TauD1=22.0 !SD,Process Lag
TauD2=24.0 !SD,Process Lag
KP11=0.1247 !SD,Process Gain
KP12=0.546 !SD,Process Gain
KP21=-0.124 !SD,Process Gain
KP22=1.116 !SD,Process Gain
KPD1=0.588 !SD,Disturbance Gain
KPD2=1.41 !SD,Disturbance Gain

(1)

!SD,Formulate the Feedback Control Loop Equations

Select Case(iRange)

Case(0)
PV1=U(1)+U(2)
If(iRes==9)Ei1(iCount)=SP1-PV1
CO1=KC1*(SP1-PV1)+Kci1*(SP1-PV1)*T

PV2=U(4)+U(3)
If(iRes==9)Ei2(iCount)=SP2-PV2
CO2=KC2*(SP2-PV2)+Kci2*(SP2-PV2)*T

F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)
F(6)=-U(6)

Case(1)

PV1=U(1)+U(2)+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-
Thetad)
If(iRes==9)Ei1(iCount)=SP1-PV1
IE1=0.0
Do i=1,iCount-1
IE1=IE1+(Ei1(i+1)+Ei1(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE1=IE1+(Ei1(iCount)+SP1-PV1)*(T-Time(iCount))/2.0
CO1=KC1*(SP1-PV1)+Kci1*IE1

(3)

KC1=32.08 !SD,Controller Gain
KC2=3.58 !SD,Controller Gain
Kci1=3.208 !SD,Integral Gain
Kci2=0.163 !SD,Integral Gain
Kcd=0.0 !SD,Derivative Gain
iCount=0 !SD,Counter for the History Profiles
SP1=0 !SD,Process Variable Set Point Value of Xd
SP2=0 !SD,Process Variable Set Point Value of Xb
DS=0.1 !SD,Disturbance Set Point Value

@Initial Conditions

U(1:Neq)=0.0

@Model Equations

Dim IE1,IE2 As Real

Dim i As Integer

!SD,Collect the History of the Integrated Model Variables

!-----
If (iRes==9)Then
iCount=iCount+1
Time(iCount)=T
Y1i(iCount)=U(1)
Y2i(iCount)=U(2)
Y3i(iCount)=U(3)
Y4i(iCount)=U(4)
Y5i(iCount)=U(5)
Y6i(iCount)=U(6)
EndIf

(2)

PV2=U(3)+U(4)+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)
If(iRes==9)Ei2(iCount)=SP2-PV2
IE2=0.0

Do i=1,iCount-1
IE2=IE2+(Ei2(i+1)+Ei2(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE2=IE2+(Ei2(iCount)+SP2-PV2)*(T-Time(iCount))/2.0
CO2=KC2*(SP2-PV2)+Kci2*IE2

F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)+KPD1*DS
F(6)=-U(6)+KPD2*DS

End Select

!SD,Collect History of the Process Variables and Control Output

!-----
If (iRes==9)Then
PV1i(iCount)=PV1
CO1i(iCount)=CO1
PV2i(iCount)=PV2
CO2i(iCount)=CO2
EndIf

(4)

@Coefficient Matrix

E(1)=Tau11
E(2)=Tau12
E(3)=Tau21
E(4)=Tau22
E(5)=TauD1
E(6)=TauD2

@Solver Options

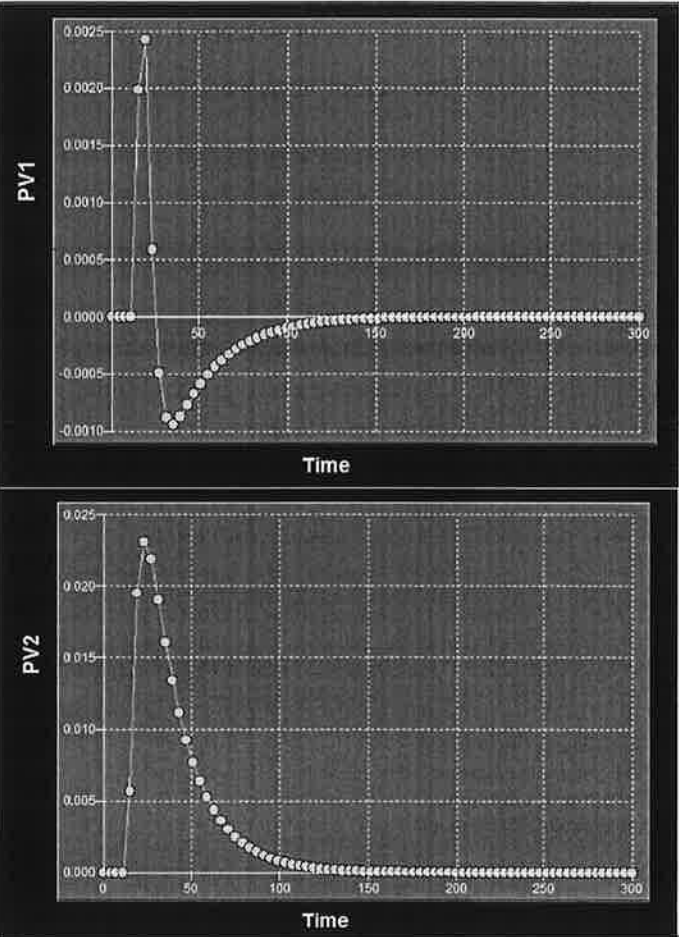
Neq=6
Npts=75
Tend=300.0
Tstop=7.0

Param(28)=1 !Continue Integration Past Dead Time
Param(177)=1 !Enable Integration History Access
VariableNames=Time;Y1;Y2;Y3;Y4;Y5;Y6 !Variable Names
Append=Pv1;Pv2 !Display Process Variable and
Control Output

@After Calling Solver

If(iCount==0)Then
Pv1=0.0
Pv2=0.0
CO1=KC1*(SP1-PV1)
CO2=KC2*(SP2-PV2)
Else
Pv1=QuadraticSpline(Time(1:iCount),Pv1i(1:iCount),Tini)
CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini)
Pv2=QuadraticSpline(Time(1:iCount),Pv2i(1:iCount),Tini)
CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini)
EndIf

(5)



Problem 2; Question 9:

```
!PID Controllers
!Regulatory-Control Implementation in Time-Domain
!Written by: Shereen Daik
!*****
Use AthenaUtilities

Global KC1,KC2,Kci1,Kci2,Kcd,SP1,SP2,DS,KP11,KP12 As Real
Global KP21,KP22,KPD1,KPD2,Tau11,Tau21,Tau12 As Real
Global Tau22,TauD1,TauD2,CO1,CO2,PV1,PV2 As Real
Global Time(5000),Y1i(5000),Y2i(5000),Y3i(5000),Y4i(5000)As
Real
Global Y5i(5000),Y6i(5000),CO1i(5000),CO2i(5000),PV1i(5000)
As Real
Global PV2i(5000),Ei1(5000),Ei2(5000),Thetap,Thetad As Real
Global iCount As Integer
```

```
Thetap=0.0    ISD,Process Dead Time
Thetad=7.0    ISD,Disturbance Dead time
Tau11=10.0    ISD,Process Lag
Tau21=16.0    ISD,Process Lag
Tau12=24.0    ISD,Process Lag
Tau22=22.0    ISD,Process Lag
TauD1=22.0    ISD,Process Lag
TauD2=24.0    ISD,Process Lag
KP11=0.1247   ISD,Process Gain
KP12=0.546    ISD,Process Gain
KP21=-0.124   ISD,Process Gain
KP22=1.116    ISD,Process Gain
KPD1=0.588    ISD,Disturbance Gain
KPD2=1.41     ISD,Disturbance Gain
```

(1)

```
!SD,Formulate the Feedback Control Loop Equations
```

```
!-----
Select Case(iRange)
```

```
Case(0)
PV1=U(1)+U(2)
If(iRes==9)Ei1(iCount)=SP1-PV1
CO1=KC1*(SP1-PV1)+Kci1*(SP1-PV1)*T

PV2=U(4)+U(3)
If(iRes==9)Ei2(iCount)=SP2-PV2
CO2=KC2*(SP2-PV2)+Kci2*(SP2-PV2)*T

F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)
F(6)=-U(6)
```

```
Case(1)
```

```
PV1=U(1)+U(2)+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-
Thetad)
If(iRes==9)Ei1(iCount)=SP1-PV1
IE1=0.0
Do i=1,iCount-1
IE1=IE1+(Ei1(i+1)+Ei1(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE1=IE1+(Ei1(iCount)+SP1-PV1)*(T-Time(iCount))/2.0
CO1=KC1*(SP1-PV1)+Kci1*IE1
```

(3)

```
KC1=32.08    ISD,Controller Gain
KC2=3.58     ISD,Controller Gain
Kci1=3.208   ISD,Integral Gain
Kci2=0.163   ISD,Integral Gain
Kcd=0.0      ISD,Derivative Gain
iCount=0     ISD,Counter for the History Profiles
SP1=0.02     ISD,Process Variable Set Point Value of Xd
SP2=0        ISD,Process Variable Set Point Value of Xb
DS=0         ISD,Disturbance Set Point Value
```

```
@Initial Conditions
U(1:Neq)=0.0
```

```
@Model Equations
Dim IE1,IE2 As Real
Dim i As Integer
```

```
!SD,Collect the History of the Integrated Model Variables
```

```
!-----
If (iRes==9)Then
iCount=iCount+1
Time(iCount)=T
Y1i(iCount)=U(1)
Y2i(iCount)=U(2)
Y3i(iCount)=U(3)
Y4i(iCount)=U(4)
Y5i(iCount)=U(5)
Y6i(iCount)=U(6)
EndIf
```

(2)

```
PV2=U(3)+U(4)+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)
If(iRes==9)Ei2(iCount)=SP2-PV2
IE2=0.0
```

```
Do i=1,iCount-1
IE2=IE2+(Ei2(i+1)+Ei2(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE2=IE2+(Ei2(iCount)+SP2-PV2)*(T-Time(iCount))/2.0
CO2=KC2*(SP2-PV2)+Kci2*IE2
```

```
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)+KPD1*DS
F(6)=-U(6)+KPD2*DS
```

```
End Select
```

```
!SD,Collect History of the Process Variables and Control Output
```

```
!-----
If (iRes==9)Then
PV1i(iCount)=PV1
CO1i(iCount)=CO1
PV2i(iCount)=PV2
CO2i(iCount)=CO2
EndIf
```

(4)

@Coefficient Matrix

E(1)=Tau11
E(2)=Tau12
E(3)=Tau21
E(4)=Tau22
E(5)=TauD1
E(6)=TauD2

@Solver Options

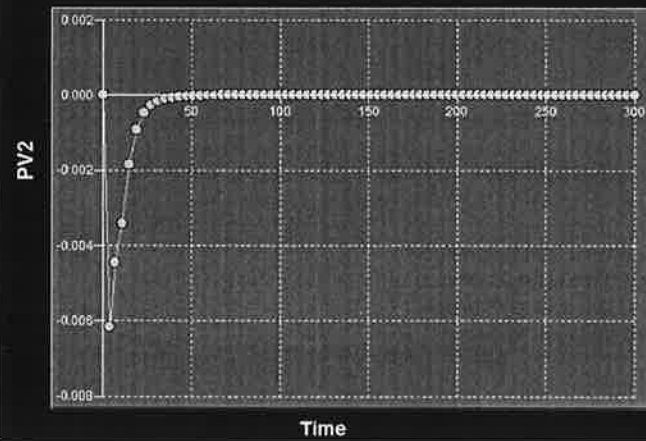
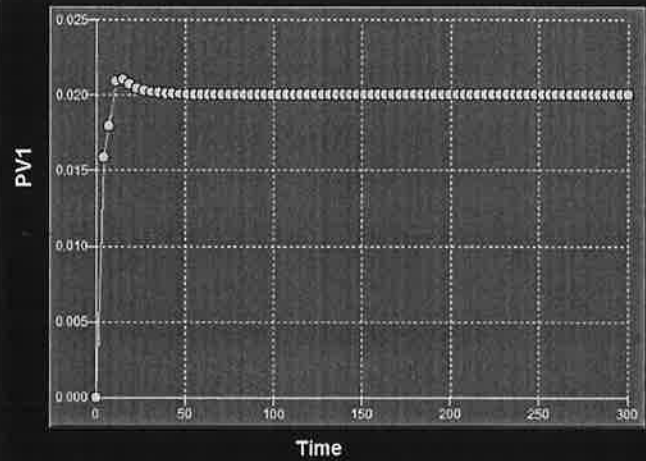
Neq=6
Npts=75
Tend=300.0
Tstop=7.0

Param(28)=1 !Continue Integration Past Dead Time
Param(177)=1 !Enable Integration History Access
VariableNames=Time;Y1;Y2;Y3;Y4;Y5;Y6 !Variable Names
Append=PV1;PV2 !Display Process Variable and Control Output

@After Calling Solver

```
If(iCount==0)Then
  PV1=0.0
  PV2=0.0
  CO1=KC1*(SP1-PV1)
  CO2=KC2*(SP2-PV2)
Else
  PV1=QuadraticSpline(Time(1:iCount),PV1i(1:iCount),Tini)
  CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini)
  PV2=QuadraticSpline(Time(1:iCount),PV2i(1:iCount),Tini)
  CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini)
EndIf
```

(5)



Problem 2; Question 10 (Complete Q8 Using Ziegler-Nichols):

!PID Controllers

!-----

!Regulatory-Control Implementation in Time-Domain

!-----

!Written by: Shereen Daik

!*****

Use AthenaUtilities

Global Tau11,Tau12,Tau21,Tau22,Tau1,Tau2 As Real

Global KC1,KC2,KC11,KC12,KCD1,KCD2,SPXD,SPXB,DS As Real

Global KP11,KP12,KP21,KP22,KP1,KP2 As Real

Global THETA11,THETA12,THETA21,THETA22,Thetap As Real

Global SustM, Ti1, Ti2,Thetad As Real

Global Time(5000),Y1i(5000),Y2i(5000 As Real

Global Y3i(5000),Y4i(5000), Y5i(5000),Y6i(5000) As Real

Global CO1i(5000),CO2i(5000),PV1i(5000),PV2i(5000) As Real

Global E1i(5000), E2i(5000) As Real

Global iCount As Integer

Tau11=10.0 !SD,Process Lag

Tau12=24.0 !SD,Process Lag

Tau21=16.0 !SD,Process Lag

Tau22=22.0 !SD,Process Lag

Tau1=22.0 !SD,Process Lag

Tau2=24.0 !SD,Process Lag

KP11=0.1247 !SD,Process Gain

KP12=0.546 !SD,Process Gain

KP21=-0.1246 !SD,Process Gain

KP22=1.116 !SD,Process Gain

KP1=0.588 !SD,Process Gain

KP2=1.41 !SD,Process Gain

(1)

THETA11=0.0 !SD,Time Delay

THETA12=0.0 !SD,Process Gain

THETA21=0.0 !SD,Process Gain

THETA22=0.0 !SD,Process Gain

Thetap=5.0 !SD,Process Gain

Thetad=7.0 !SD,Process Gain

KC1=32.08 !SD,Proportional Gain

KC11=0.1 !SD,Integral Gain

KCD1=0.0 !SD,Derivative Gain

KC2=3.58 !SD,Proportional Gain

KC12=0.1 !SD,Integral Gain

KCD2=0.0 !SD,Derivative Gain

SustM=2.0 !SD,Stability Margin

!SD,Ziegler-Nichols Controller Constants

!-----

If(KC1>0.0 .AND. KCD1==0.0)Then !SD,Proportional-Integral Controller

KC1=0.9/(SustM*KP11)*Tau11/Thetad

Ti1=3.33*Thetad

KC11=KC1/Ti1

EndIf

If(KC12>0.0 .AND. KCD2==0.0)Then !SD,Proportional-Integral Controller

KC2=0.9/(SustM*KP22)*Tau22/Thetad

Ti2=3.33*Thetad

KC12=KC2/Ti2

EndIf

iCount=0

SPXD=0.0

SPXB=0.0

DS=0.1

(2)

@Initial Conditions

U(1:Neq)=0.0

@Model Equations

Dim PV1,PV2,CO1,CO2,IE As Real

Dim i As Integer

ISD,Collect the History of the Integrated Model Variables

If (iRes==9)Then

iCount=iCount+1

Time(iCount)=T

Y1i(iCount)=U(1)

Y2i(iCount)=U(2)

Y3i(iCount)=U(3)

Y4i(iCount)=U(4)

Y5i(iCount)=U(5)

Y6i(iCount)=U(6)

EndIf

Case(1)

PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)

PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)

If(iRes==9)E1i(iCount)=SPXD-PV1

IE=0.0

Do i=1, iCount-1

IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0

EndDo

IE=IE+(E1i(iCount)+SPXD-PV1)*(T-Time(iCount))/2.0

CO1=KC1*(SPXD-PV1)+KCI1*IE

F(1)=-U(1)+KP11*CO1

F(2)=-U(2)+KP12*CO2

F(3)=-U(3)+KP21*CO1

F(4)=-U(4)+KP22*CO2

F(5)=-U(5)

F(6)=-U(6)

If(iRes==9)E2i(iCount)=SPXB-PV2

IE=0.0

Do i=1, iCount-1

IE=IE+(E2i(i+1)+E2i(i))*(Time(i+1)-Time(i))/2.0

EndDo

IE=IE+(E2i(iCount)+SPXB-PV2)*(T-Time(iCount))/2.0

CO2=KC2*(SPXB-PV2)+KCI2*IE

F(1)=-U(1)+KP11*CO1

F(2)=-U(2)+KP12*CO2

F(3)=-U(3)+KP21*CO1

F(4)=-U(4)+KP22*CO2

F(5)=-U(5)

F(6)=-U(6)

Case(2)

PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)

(3)

ISD,Formulate the Feedback Control Loop Equations

Select Case(iRange)

Case(0)

PV1=U(1)+U(2)+U(5)

PV2=U(3)+U(4)+U(6)

If(iRes==0)E1i(iCount)=SPXD-PV1

CO1=KC1*(SPXD-PV1)+KCI1*(SPXD-PV1)*T

F(1)=-U(1)

F(2)=-U(2)

F(3)=-U(3)

F(4)=-U(4)

F(5)=-U(5)

F(6)=-U(6)

If(iRes==0)E2i(iCount)=SPXB-PV2

CO2=KC2*(SPXB-PV2)+KCI2*(SPXB-PV2)*T

F(1)=-U(1)

F(2)=-U(2)

F(3)=-U(3)

F(4)=-U(4)

F(5)=-U(5)

F(6)=-U(6)

F(1)=-U(1)+KP11*CO1

F(2)=-U(2)+KP12*CO2

F(3)=-U(3)+KP21*CO1

F(4)=-U(4)+KP22*CO2

F(5)=-U(5)+KP1*DS

F(6)=-U(6)+KP2*DS

If(iRes==9)E2i(iCount)=SPXB-PV2

IE=0.0

Do i=1, iCount-1

IE=IE+(E2i(i+1)+E2i(i))*(Time(i+1)-Time(i))/2.0

EndDo

IE=IE+(E2i(iCount)+SPXB-PV2)*(T-Time(iCount))/2.0

CO2=KC2*(SPXB-PV2)+KCI2*IE

F(1)=-U(1)+KP11*CO1

F(2)=-U(2)+KP12*CO2

F(3)=-U(3)+KP21*CO1

F(4)=-U(4)+KP22*CO2

F(5)=-U(5)+KP1*DS

F(6)=-U(6)+KP2*DS

End Select

ISD,Collect History of the Process Variables and Control Output

If (iRes==9)Then

PV1i(iCount)=PV1

CO1i(iCount)=CO1

PV2i(iCount)=PV2

CO2i(iCount)=CO2

EndIf

(4)

```
PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)
```

```
If(iRes==9)E1i(iCount)=SPXD-PV1
IE=0.0
Do i=1, iCount-1
IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(E1i(iCount)+SPXD-PV1)*(T-Time(iCount))/2.0
CO1=KC1*(SPXD-PV1)+KC11*IE
```

```
@After Calling Solver
Dim PV1,PV2,CO1,CO2 As Real
If(iCount==0)Then
PV1=U(1)+U(2)+U(5)
CO1=KC1*(SPXD-PV1)
PV2=U(3)+U(4)+U(6)
CO2=KC2*(SPXB-PV2)
Else
PV1=QuadraticSpline(Time(1:iCount),PV1i(1:iCount),Tini)
CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini)
PV2=QuadraticSpline(Time(1:iCount),PV2i(1:iCount),Tini)
CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini)
EndIf
```

(5)

```
@Coefficient Matrix
```

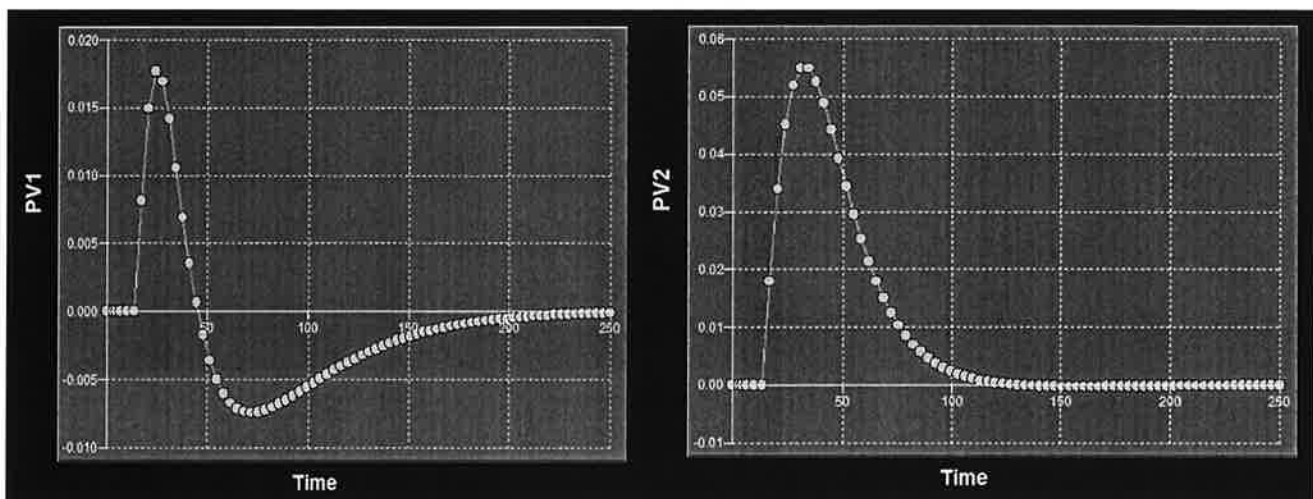
```
E(1)=Tau11
E(2)=Tau12
E(3)=Tau21
E(4)=Tau22
E(5)=Tau1
E(6)=Tau2
```

```
@Solver Options
```

```
Neq=6
Npts=75
Tend=250.0
Tstop=5;7
```

```
Param(28)=1          ISD,Continue Integration Past Dead Time
Param(177)=1         ISD,Enable Integration History Access
VariableNames=Time;Y1;Y2  ISD,Variable Names
Append=PV1;CO1;PV2;CO2    ISD,Display Process Variable and Control
Output
```

(6)



Problem 2; Question 10 (Complete Q8 Using Ziegler-Nichols Ultimate Gain) :

!PID Controllers

!-----

!Regulatory-Control Implementation in Time-Domain

!-----

!Written by: Shereen Daik

!*****

Use AthenaUtilities

Global Tau11,Tau12,Tau21,Tau22,Tau1,Tau2 As Real

Global KC1,KC2,KC11,KC12,KCD1,KCD2,SPXD,SPXB,DS As Real

Global KP11,KP12,KP21,KP22,KP1,KP2 As Real

Global THETA11,THETA12,THETA21,THETA22,Thetap As Real

Global SustM, Ti1, Ti2,Thetad, KU1,KU2,PU1,PU2 As Real

Global Time(5000),Y1i(5000),Y2i(5000 As Real

Global Y3i(5000),Y4i(5000), Y5i(5000),Y6i(5000) As Real

Global CO1i(5000),CO2i(5000),PV1i(5000),PV2i(5000) As Real

Global E1i(5000), E2i(5000) As Real

Global iCount As Integer

Tau11=10.0 !SD,Process Lag

Tau12=24.0 !SD,Process Lag

Tau21=16.0 !SD,Process Lag

Tau22=22.0 !SD,Process Lag

Tau1=22.0 !SD,Process Lag

Tau2=24.0 !SD,Process Lag

KP11=0.1247 !SD,Process Gain

KP12=0.546 !SD,Process Gain

KP21=-0.1246 !SD,Process Gain

KP22=1.116 !SD,Process Gain

KP1=0.588 !SD,Process Gain

KP2=1.41 !SD,Process Gain

(1)

THETA11=0.0 !SD,Time Delay

THETA12=0.0 !SD,Process Gain

THETA21=0.0 !SD,Process Gain

THETA22=0.0 !SD,Process Gain

Thetap=5.0 !SD,Process Gain

Thetad=7.0 !SD,Process Gain

KC1=32.08 !SD,Proportional Gain

KC11=0.1 !SD,Integral Gain

KCD1=0.0 !SD,Derivative Gain

KC2=3.58 !SD,Proportional Gain

KC12=0.1 !SD,Integral Gain

KCD2=0.0 !SD,Derivative Gain

SustM=2.0 !SD,Stability Margin

KU1=30.53 !SD, Ultimate Gain

KU2=6.78 !SD, Ultimate Gain

PU1=17.10 !SD,Ultimate Period

PU2=18.43 !SD,Ultimate Period

!SD,Ziegler-Nichols Controller Constants

!-----

If(KC1>0.0 .AND. KCD1==0.0)Then !SD,Proportional-Integral
Controller

KC1=0.45*KU1/SustM

Ti1=0.83*PU1

KC11=KC1/Ti1

EndIf

If(KC12>0.0 .AND. KCD2==0.0)Then !SD,Proportional-Integral
Controller

KC2=0.45*KU2/SustM

Ti2=0.83*PU2

KC12=KC2/Ti2

EndIf

iCount=0

SPXD=0.0

SPXB=0.0

DS=0.1

(2)

@Initial Conditions

U(1:Neq)=0.0

@Model Equations

Dim PV1,PV2,CO1,CO2,IE As Real

Dim i As Integer

!SD,Collect the History of the Integrated Model Variables

If (iRes==9)Then

iCount=iCount+1

Time(iCount)=T

Y1i(iCount)=U(1)

Y2i(iCount)=U(2)

Y3i(iCount)=U(3)

Y4i(iCount)=U(4)

Y5i(iCount)=U(5)

Y6i(iCount)=U(6)

EndIf

Case(1)

PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)

PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)

If(iRes==9)E1i(iCount)=SPXD-PV1

IE=0.0

Do i=1, iCount-1

IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0

EndDo

IE=IE+(E1i(iCount)+SPXD-PV1)*(T-Time(iCount))/2.0

CO1=KC1*(SPXD-PV1)+KCI1*IE

F(1)=-U(1)+KP11*CO1

F(2)=-U(2)+KP12*CO2

F(3)=-U(3)+KP21*CO1

F(4)=-U(4)+KP22*CO2

F(5)=-U(5)

F(6)=-U(6)

If(iRes==9)E2i(iCount)=SPXB-PV2

IE=0.0

Do i=1, iCount-1

IE=IE+(E2i(i+1)+E2i(i))*(Time(i+1)-Time(i))/2.0

EndDo

IE=IE+(E2i(iCount)+SPXB-PV2)*(T-Time(iCount))/2.0

CO2=KC2*(SPXB-PV2)+KCI2*IE

F(1)=-U(1)+KP11*CO1

F(2)=-U(2)+KP12*CO2

F(3)=-U(3)+KP21*CO1

F(4)=-U(4)+KP22*CO2

F(5)=-U(5)

F(6)=-U(6)

Case(2)

PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)

(3)

!SD,Formulate the Feedback Control Loop Equations

Select Case(iRange)

Case(0)

PV1=U(1)+U(2)+U(5)

PV2=U(3)+U(4)+U(6)

If(iRes==0)E1i(iCount)=SPXD-PV1

CO1=KC1*(SPXD-PV1)+KCI1*(SPXD-PV1)*T

F(1)=-U(1)

F(2)=-U(2)

F(3)=-U(3)

F(4)=-U(4)

F(5)=-U(5)

F(6)=-U(6)

If(iRes==0)E2i(iCount)=SPXB-PV2

CO2=KC2*(SPXB-PV2)+KCI2*(SPXB-PV2)*T

F(1)=-U(1)

F(2)=-U(2)

F(3)=-U(3)

F(4)=-U(4)

F(5)=-U(5)

F(6)=-U(6)

F(1)=-U(1)+KP11*CO1

F(2)=-U(2)+KP12*CO2

F(3)=-U(3)+KP21*CO1

F(4)=-U(4)+KP22*CO2

F(5)=-U(5)+KP1*DS

F(6)=-U(6)+KP2*DS

If(iRes==9)E2i(iCount)=SPXB-PV2

IE=0.0

Do i=1, iCount-1

IE=IE+(E2i(i+1)+E2i(i))*(Time(i+1)-Time(i))/2.0

EndDo

IE=IE+(E2i(iCount)+SPXB-PV2)*(T-Time(iCount))/2.0

CO2=KC2*(SPXB-PV2)+KCI2*IE

F(1)=-U(1)+KP11*CO1

F(2)=-U(2)+KP12*CO2

F(3)=-U(3)+KP21*CO1

F(4)=-U(4)+KP22*CO2

F(5)=-U(5)+KP1*DS

F(6)=-U(6)+KP2*DS

End Select

!SD,Collect History of the Process Variables and Control Output

If (iRes==9)Then

PV1i(iCount)=PV1

CO1i(iCount)=CO1

PV2i(iCount)=PV2

CO2i(iCount)=CO2

EndIf

(4)

```
PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)
```

```
If(iRes==9)E1i(iCount)=SPXD-PV1
```

```
IE=0.0
```

```
Do i=1, iCount-1
```

```
IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0
```

```
EndDo
```

```
IE=IE+(E1i(iCount)+SPXD-PV1)*(T-Time(iCount))/2.0
```

```
CO1=KC1*(SPXD-PV1)+KCI1*IE
```

```
@After Calling Solver
```

```
Dim PV1,PV2,CO1,CO2 As Real
```

```
If(iCount==0)Then
```

```
PV1=U(1)+U(2)+U(5)
```

```
CO1=KC1*(SPXD-PV1)
```

```
PV2=U(3)+U(4)+U(6)
```

```
CO2=KC2*(SPXB-PV2)
```

```
Else
```

```
PV1=QuadraticSpline(Time(1:iCount),PV1i(1:iCount),Tini)
```

```
CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini)
```

```
PV2=QuadraticSpline(Time(1:iCount),PV2i(1:iCount),Tini)
```

```
CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini)
```

```
EndIf
```

(5)

```
@Coefficient Matrix
```

```
E(1)=Tau11
```

```
E(2)=Tau12
```

```
E(3)=Tau21
```

```
E(4)=Tau22
```

```
E(5)=Tau1
```

```
E(6)=Tau2
```

```
@Solver Options
```

```
Neq=6
```

```
Npts=75
```

```
Tend=250.0
```

```
Tstop=5;7
```

```
Param(28)=1
```

```
!SD,Continue Integration Past Dead Time
```

```
Param(177)=1
```

```
!SD,Enable Integration History Access
```

```
VariableNames=Time;Y1;Y2
```

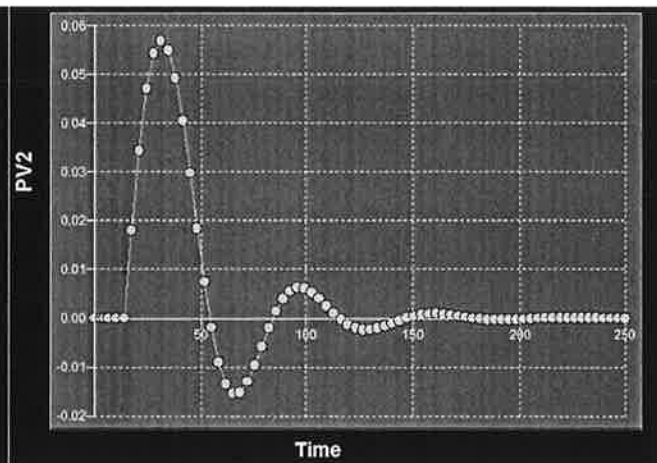
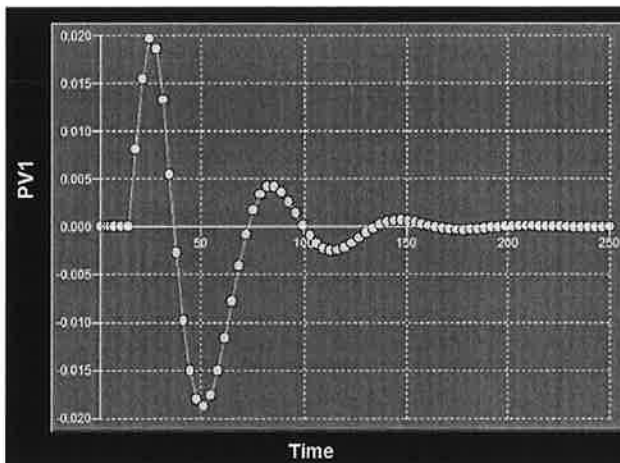
```
!SD,Variable Names
```

```
Append=PV1;CO1;PV2;CO2
```

```
!SD,Display Process Variable and
```

```
Control Output
```

(6)



Problem 2; Question 10 (Complete Q8 Using Cohen-Coon) :

IPID Controllers

Regulatory-Control Implementation in Time-Domain

Written by: Shereen Daik

Use AthenaUtilities

Global Tau11,Tau12,Tau21,Tau22,Tau1,Tau2 As Real

Global KC1,KC2,KC11,KC12,KCD1,KCD2,SPXD,SPXB,DS As Real

Global KP11,KP12,KP21,KP22,KP1,KP2 As Real

Global THETA11,THETA12,THETA21,THETA22,Thetap As Real

Global SustM, Ti1, Ti2,Thetad, KU1,KU2,PU1,PU2 As Real

Global Time(5000),Y1i(5000),Y2i(5000 As Real

Global Y3i(5000),Y4i(5000), Y5i(5000),Y6i(5000) As Real

Global CO1i(5000),CO2i(5000),PV1i(5000),PV2i(5000) As Real

Global E1i(5000), E2i(5000) As Real

Global iCount As Integer

Tau11=10.0 ISD,Process Lag

Tau12=24.0 ISD,Process Lag

Tau21=16.0 ISD,Process Lag

Tau22=22.0 ISD,Process Lag

Tau1=22.0 ISD,Process Lag

Tau2=24.0 ISD,Process Lag

KP11=0.1247 ISD,Process Gain

KP12=0.546 ISD,Process Gain

KP21=-0.1246 ISD,Process Gain

KP22=1.116 ISD,Process Gain

KP1=0.588 ISD,Process Gain

KP2=1.41 ISD,Process Gain

(1)

THETA11=0.0 ISD,Time Delay

THETA12=0.0 ISD,Process Gain

THETA21=0.0 ISD,Process Gain

THETA22=0.0 ISD,Process Gain

Thetap=5.0 ISD,Process Gain

Thetad=7.0 ISD,Process Gain

KC1=32.08 ISD,Proportional Gain

KC11=0.1 ISD,Integral Gain

KCD1=0.0 ISD,Derivative Gain

KC2=3.58 ISD,Proportional Gain

KC12=0.1 ISD,Integral Gain

KCD2=0.0 ISD,Derivative Gain

SustM=2.0 ISD,Stability Margin

KU1=30.53 ISD, Ultimate Gain

KU2=6.78 ISD, Ultimate Gain

PU1=17.10 ISD,Ultimate Period

PU2=18.43 ISD,Ultimate Period

ISD,Ziegler-Nichols Controller Constants

If(KC11>0.0 .AND. KCD1==0.0)Then ISD,Proportional-Integral
Controller

KC1=0.9/(SustM*KP11)*(Tau11/Thetad+0.092)

Ti1=3.33*Thetad*((Tau11+0.092*Thetad)/(Tau11+2.22*Thetad))

KC11=KC1/Ti1

EndIf

If(KC12>0.0 .AND. KCD2==0.0)Then ISD,Proportional-Integral
Controller

KC2=0.9/(SustM*KP22)*(Tau22/Thetad+0.092)

Ti2=3.33*Thetad*((Tau22+0.092*Thetad)/(Tau22+2.22*Thetad))

KC12=KC2/Ti2

EndIf

iCount=0

SPXD=0.0

SPXB=0.0

DS=0.1

(2)

@Initial Conditions
U(1:Neq)=0.0

@Model Equations
Dim PV1,PV2,CO1,CO2,IE As Real
Dim i As Integer

ISD,Collect the History of the Integrated Model Variables

```
If (iRes==9)Then
  iCount=iCount+1
  Time(iCount)=T
  Y1(iCount)=U(1)
  Y2(iCount)=U(2)
  Y3(iCount)=U(3)
  Y4(iCount)=U(4)
  Y5(iCount)=U(5)
  Y6(iCount)=U(6)
EndIf

Case(1)
  PV1=QuadraticSpline(Time(1:iCount),Y1(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y2(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)
  PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)
```

```
If(iRes==9)E1(iCount)=SPXD-PV1
IE=0.0
Do i=1, iCount-1
  IE=IE+(E1(i+1)+E1(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(E1(iCount)+SPXD-PV1)*(T-Time(iCount))/2.0
CO1=KC1*(SPXD-PV1)+KC11*IE
```

```
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)
F(6)=-U(6)
```

```
If(iRes==9)E2(iCount)=SPXB-PV2
IE=0.0
Do i=1, iCount-1
  IE=IE+(E2(i+1)+E2(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(E2(iCount)+SPXB-PV2)*(T-Time(iCount))/2.0
CO2=KC2*(SPXB-PV2)+KC12*IE
```

```
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)
F(6)=-U(6)
```

```
Case(2)
  PV1=QuadraticSpline(Time(1:iCount),Y1(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)
```

(3)

ISD,Formulate the Feedback Control Loop Equations

Select Case(iRange)

```
Case(0)
  PV1=U(1)+U(2)+U(5)
  PV2=U(3)+U(4)+U(6)

  If(iRes==0)E1(iCount)=SPXD-PV1
  CO1=KC1*(SPXD-PV1)+KC11*(SPXD-PV1)*T
  F(1)=-U(1)
  F(2)=-U(2)
  F(3)=-U(3)
  F(4)=-U(4)
  F(5)=-U(5)
  F(6)=-U(6)
```

```
If(iRes==0)E2(iCount)=SPXB-PV2
  CO2=KC2*(SPXB-PV2)+KC12*(SPXB-PV2)*T
  F(1)=-U(1)
  F(2)=-U(2)
  F(3)=-U(3)
  F(4)=-U(4)
  F(5)=-U(5)
  F(6)=-U(6)
```

```
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)+KP1*DS
F(6)=-U(6)+KP2*DS
```

```
If(iRes==9)E2(iCount)=SPXB-PV2
IE=0.0
Do i=1, iCount-1
  IE=IE+(E2(i+1)+E2(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(E2(iCount)+SPXB-PV2)*(T-Time(iCount))/2.0
CO2=KC2*(SPXB-PV2)+KC12*IE
```

```
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)+KP1*DS
F(6)=-U(6)+KP2*DS
```

End Select

ISD,Collect History of the Process Variables and Control Output

```
If (iRes==9)Then
  PV1(iCount)=PV1
  CO1(iCount)=CO1
  PV2(iCount)=PV2
  CO2(iCount)=CO2
EndIf
```

(4)

```
PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)
```

```
If(iRes==9)E1i(iCount)=SPXD-PV1
IE=0.0
Do i=1, iCount-1
IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(E1i(iCount)+SPXD-PV1)*(T-Time(iCount))/2.0
CO1=KC1*(SPXD-PV1)+KC1*IE
```

@After Calling Solver

```
Dim PV1,PV2,CO1,CO2 As Real
If(iCount==0)Then
PV1=U(1)+U(2)+U(5)
CO1=KC1*(SPXD-PV1)
PV2=U(3)+U(4)+U(6)
CO2=KC2*(SPXB-PV2)
Else
PV1=QuadraticSpline(Time(1:iCount),PV1i(1:iCount),Tini)
CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini)
PV2=QuadraticSpline(Time(1:iCount),PV2i(1:iCount),Tini)
CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini)
EndIf
```

(5)

@Coefficient Matrix

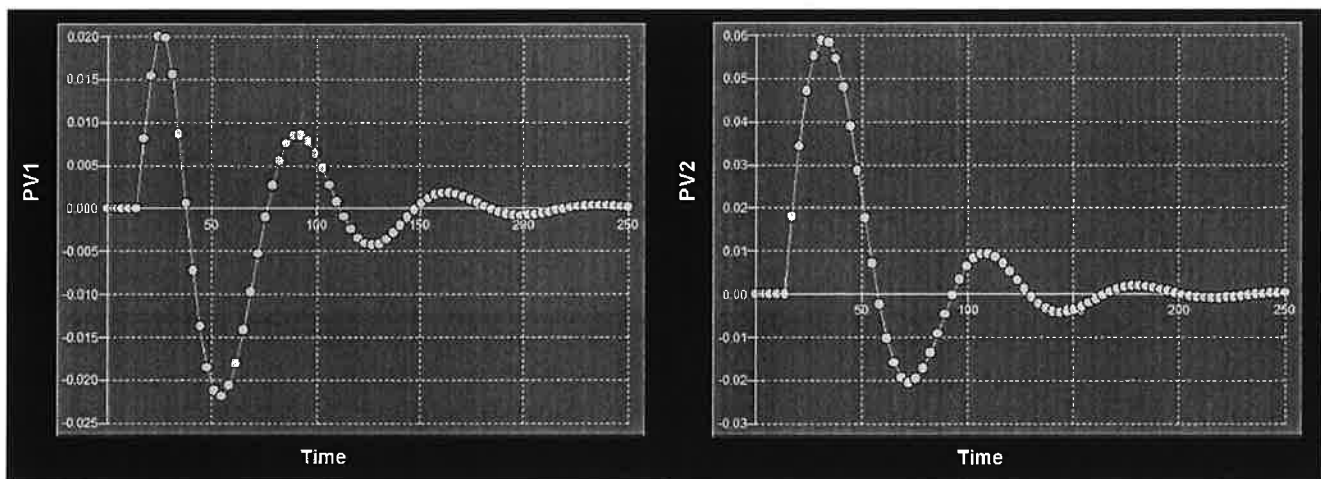
```
E(1)=Tau11
E(2)=Tau12
E(3)=Tau21
E(4)=Tau22
E(5)=Tau1
E(6)=Tau2
```

@Solver Options

```
Neq=6
Npts=75
Tend=250.0
Tstop=5;7
```

```
Param(28)=1          !SD,Continue Integration Past Dead Time
Param(177)=1         !SD,Enable Integration History Access
VariableNames=Time;Y1;Y2 !SD,Variable Names
Append=PV1;CO1;PV2;CO2 !SD,Display Process Variable and
Control Output
```

(6)



Conclusion:

For this problem, it appears as though the Ziegler-Nicholos is the best tuning method, since it gives the least number of oscillations compared to Ziegler-Nicholos Ultimate Gain and Cohen-Coon.

Problem 2; Question 10 (Complete Q9 Using Ziegler-Nicholos :

```

!PID Controllers
!-----
!Regulatory-Control Implementation in Time-Domain
!-----
!Written by: Shereen Daik
!*****
Use AthenaUtilities
Global Tau11,Tau12,Tau21,Tau22,Tau1,Tau2 As Real
Global KC1,KC2,KC11,KC12,KCD1,KCD2,SPXD,SPXB,DS As Real
Global KP11,KP12,KP21,KP22,KP1,KP2 As Real
Global THETA11,THETA12,THETA21,THETA22,Thetap As Real
Global StabM, Ti1, Ti2,Thetad, KU1,KU2,PU1,PU2 As Real
Global Time(5000),Y1i(5000),Y2i(5000 As Real
Global Y3i(5000),Y4i(5000), Y5i(5000),Y6i(5000) As Real
Global CO1i(5000),CO2i(5000),PV1i(5000),PV2i(5000) As Real
Global E1i(5000), E2i(5000) As Real
Global iCount As Integer

Thetap=0.0   !SD,Process Dead Time
Thetad=7.0   !SD,Disturbance Dead time
Tau11=10.0   !SD,Process Lag Time
Tau21=16.0   !SD,Process Lag Time
Tau12=24.0   !SD,Process Lag Time
Tau22=22.0   !SD,Process Lag Time
TauD1=22.0   !SD,Disturbance Lag Time
TauD2=24.0   !SD,Disturbance Lag Time
KP11=0.1247  !SD,Process Gain
KP12=0.546   !SD,Process Gain
KP21=-0.1246 !SD,Process Gain
KP22=1.116   !SD,Process Gain
KPD1=0.588   !SD,Disturbance Gain
KPD2=1.41    !SD,Disturbance Gain

StabM=2.0    !SD,Stability Margin

KC1=32.08    !SD,Proportional Gain
KC2=3.58     !SD,Proportional Gain
Kci1=3.208   !SD,Integral Gain
Kci2=0.163   !SD,Integral Gain
Kcd1=0.0     !SD,Derivative Gain
Kcd2=0.0     !SD,Derivative Gain

!SD,Ziegler-Nichols Controller Constants
!-----
If(Kci1>0.0 .AND. Kcd1==0.0)Then !SD,Proportional-Integral
Controller
KC1=0.9/(StabM*KP11)*Tau11/Thetad
Ti1=3.33*Thetad
Kci1=KC1/Ti1
EndIf

If(Kci2>0.0 .AND. Kcd2==0.0)Then !SD,Proportional-Integral
Controller
KC2=0.9/(StabM*KP22)*Tau22/Thetad
Ti2=3.33*Thetad
Kci2=KC2/Ti2
EndIf

```

(1)

```

iCount=0   !SD,Counter for the History Profiles
SP1=0.02   !SD,Process Variable Set Point Value of Xd
SP2=0.0    !SD,Process Variable Set Point Value of Xb
DS=0       !SD,Disturbance Set Point Value!
@Initial Conditions
U(1:Neq)=0.0

@Model Equations
Dim IE As Real
Dim i As Integer

!SD,Collect the History of the Integrated Model Variables
!-----
If (iRes==9)Then
iCount=iCount+1
Time(iCount)=T
Y1i(iCount)=U(1)
Y2i(iCount)=U(2)
Y3i(iCount)=U(3)
Y4i(iCount)=U(4)
Y5i(iCount)=U(5)
Y6i(iCount)=U(6)
EndIf

!SD,Formulate the Feedback Control Loop Equations
!-----
Select Case(iRange)
Case(0)
PV1=U(1)+U(2)+U(5)
PV2=U(3)+U(4)+U(6)

If(iRes==0)Ei1(iCount)=SP1-PV1
CO1=KC1*(SP1-PV1)+Kci1*(SP1-PV1)*T
F(1)=-U(1)
F(2)=-U(2)
F(3)=-U(3)
F(4)=-U(4)
F(5)=-U(5)
F(6)=-U(6)

If(iRes==0)Ei2(iCount)=SP2-PV2
CO2=KC2*(SP2-PV2)+Kci2*(SP2-PV2)*T
F(1)=-U(1)
F(2)=-U(2)
F(3)=-U(3)
F(4)=-U(4)
F(5)=-U(5)
F(6)=-U(6)

Case(1)
PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)
PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)

```

(2)

```

If(iRes==9)Ei1(iCount)=SP1-PV1
IE=0.0
Do i=1,iCount-1
    IE=IE+(Ei1(i+1)+Ei1(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(Ei1(iCount)+SP1-PV1)*(T-Time(iCount))/2.0
CO1=KC1*(SP1-PV1)+Kci1*IE
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)
F(6)=-U(6)

```

```

If(iRes==9)Ei2(iCount)=SP2-PV2
IE=0.0
Do i=1,iCount-1
    IE=IE+(Ei2(i+1)+Ei2(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(Ei2(iCount)+SP2-PV2)*(T-Time(iCount))/2.0
CO2=KC2*(SP2-PV2)+Kci2*IE
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)
F(6)=-U(6)

```

(3)

```

If(iRes==9)Ei2(iCount)=SP2-PV2
IE=0.0
Do i=1,iCount-1
    IE=IE+(Ei2(i+1)+Ei2(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(Ei2(iCount)+SP2-PV2)*(T-Time(iCount))/2.0
CO2=KC2*(SP2-PV2)+Kci2*IE
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)+KPD1*DS
F(6)=-U(6)+KPD2*DS

```

End Select

!SD,Collect History of the Process Variables and Control Output

```

If (iRes==9)Then
    PV1i(iCount)=PV1
    CO1i(iCount)=CO1
    PV2i(iCount)=PV2
    CO2i(iCount)=CO2
EndIf

```

@Coefficient Matrix

```

E(1)=Tau11
E(2)=Tau12
E(3)=Tau21
E(4)=Tau22
E(5)=TauD1
E(6)=TauD2

```

(5)

```

Case(2)
    PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-Thetap)
    +QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap)
    +QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)
    PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)
    +QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)
    +QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)

```

```

If(iRes==9)Ei1(iCount)=SP1-PV1
IE=0.0
Do i=1,iCount-1
    IE=IE+(Ei1(i+1)+Ei1(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(Ei1(iCount)+SP1-PV1)*(T-Time(iCount))/2.0
CO1=KC1*(SP1-PV1)+Kci1*IE
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)+KPD1*DS
F(6)=-U(6)+KPD2*DS

```

(4)

@Solver Options

```

Neq=6
Npts=75
Tend=300.0
Tstop=7.0

```

```

Param(28)=1          !SD,Continue Integration Past Dead Time
Param(177)=1         !SD,Enable Integration History Access
VariableNames=Time;Y1;Y2;Y3;Y4;Y5;Y6 !SD,Variable Names
Append=PV1;PV2       !SD,Display Process Variable and
Control Output

```

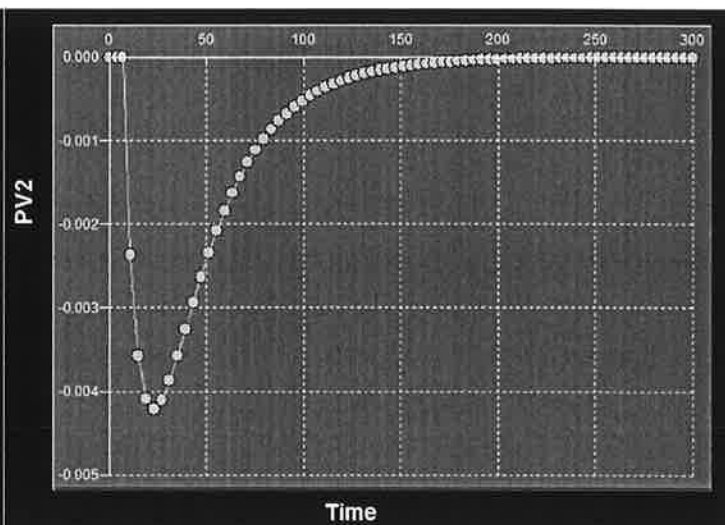
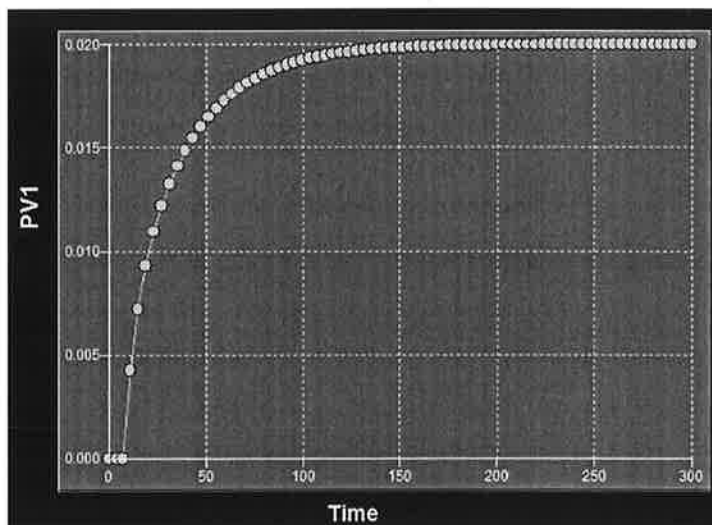
@After Calling Solver

```

If(iCount==0)Then
    PV1=0.0
    PV2=0.0
    CO1=KC1*(SP1-PV1)
    CO2=KC2*(SP2-PV2)
Else
    PV1=QuadraticSpline(Time(1:iCount),PV1i(1:iCount),Tini)
    CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini)
    PV2=QuadraticSpline(Time(1:iCount),PV2i(1:iCount),Tini)
    CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini)
EndIf

```

(6)



Problem 2; Question 10 (Complete Q9 Using Ziegler Nichols Ultimate Gain :

!PID Controllers

!Regulatory-Control Implementation in Time-Domain

!Written by: Shereen Daik

Use AthenaUtilities

```
Global Tau11,Tau12,Tau21,Tau22,Tau1,Tau2 As Real
Global KC1,KC2,KC11,KC12,KCD1,KCD2,SPXD,SPXB,DS As Real
Global KP11,KP12,KP21,KP22,KP1,KP2 As Real
Global THETA11,THETA12,THETA21,THETA22,Thetap As Real
Global StabM, Ti1, Ti2,Thetad, KU1,KU2,PU1,PU2 As Real
Global Time(5000),Y1i(5000),Y2i(5000 As Real
Global Y3i(5000),Y4i(5000), Y5i(5000),Y6i(5000) As Real
Global CO1i(5000),CO2i(5000),PV1i(5000),PV2i(5000) As Real
Global E1i(5000), E2i(5000) As Real
Global iCount As Integer
```

```
Thetap=0.0 !SD,Process Dead Time
Thetad=7.0 !SD,Disturbance Dead time
Tau11=10.0 !SD,Process Lag Time
Tau21=16.0 !SD,Process Lag Time
Tau12=24.0 !SD,Process Lag Time
Tau22=22.0 !SD,Process Lag Time
TauD1=22.0 !SD,Disturbance Lag Time
TauD2=24.0 !SD,Disturbance Lag Time
KP11=0.1247 !SD,Process Gain
KP12=0.546 !SD,Process Gain
KP21=-0.1246!SD,Process Gain
KP22=1.116 !SD,Process Gain
KPD1=0.588 !SD,Disturbance Gain
KPD2=1.41 !SD,Disturbance Gain
```

```
StabM=2.0 !SD,Stability Margin
Ku1=30.53 !SD,Ultimate Gain
Ku2=6.78 !SD,Ultimate Gain
Pu1=17.10 !SD,Ultimate Period
Pu2=18.43 !SD,Ultimate Period
```

(1)

```
KC1=32.08 !SD,Proportional Gain
KC2=3.58 !SD,Proportional Gain
Kci1=3.208 !SD,Integral Gain
Kci2=0.163 !SD,Integral Gain
Kcd1=0.0 !SD,Derivative Gain
Kcd2=0.0 !SD,Derivative Gain
```

!SD,Ziegler-Nichols Ultimate Gain

```
!If(Kci1>0.0 .AND. Kcd1==0.0)Then !SD,Proportional-Integral Controller
KC1=0.45*Ku1/StabM
Ti1=0.83*Pu1
Kci1=KC1/Ti1
EndIf
```

```
!If(Kci2>0.0 .AND. Kcd2==0.0)Then !SD,Proportional-Integral Controller
KC2=0.45*Ku2/StabM
Ti2=0.83*Pu2
Kci2=KC2/Ti2
EndIf
```

```
iCount=0 !SD,Counter for the History Profiles
SP1=0.02 !SD,Process Variable Set Point Value of Xd
SP2=0.0 !SD,Process Variable Set Point Value of Xb
DS=0 !SD,Disturbance Set Point Value
```

@Initial Conditions

```
U(1:Neq)=0.0
```

@Model Equations

```
Dim IE As Real
Dim i As Integer
```

(2)

ISD,Collect the History of the Integrated Model Variables

```

If (iRes==9)Then
  iCount=iCount+1
  Time(iCount)=T
  Y1i(iCount)=U(1)
  Y2i(iCount)=U(2)
  Y3i(iCount)=U(3)
  Y4i(iCount)=U(4)
  Y5i(iCount)=U(5)
  Y6i(iCount)=U(6)
EndIf

```

ISD,Formulate the Feedback Control Loop Equations

Select Case(iRange)

Case(0)

```

PV1=U(1)+U(2)+U(5)
PV2=U(3)+U(4)+U(6)

```

```

If(iRes==0)Ei1(iCount)=SP1-PV1
CO1=KC1*(SP1-PV1)+Kci1*(SP1-PV1)*T
F(1)=-U(1)
F(2)=-U(2)
F(3)=-U(3)
F(4)=-U(4)
F(5)=-U(5)
F(6)=-U(6)

```

```

If(iRes==0)Ei2(iCount)=SP2-PV2
CO2=KC2*(SP2-PV2)+Kci2*(SP2-PV2)*T
F(1)=-U(1)
F(2)=-U(2)
F(3)=-U(3)
F(4)=-U(4)
F(5)=-U(5)
F(6)=-U(6)

```

(3)

Case(1)

```

PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)
PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)

```

```

If(iRes==9)Ei1(iCount)=SP1-PV1
IE=0.0
Do i=1,iCount-1
  IE=IE+(Ei1(i+1)+Ei1(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(Ei1(iCount)+SP1-PV1)*(T-Time(iCount))/2.0
CO1=KC1*(SP1-PV1)+Kci1*IE
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)
F(6)=-U(6)

```

```

If(iRes==9)Ei2(iCount)=SP2-PV2
IE=0.0
Do i=1,iCount-1
  IE=IE+(Ei2(i+1)+Ei2(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(Ei2(iCount)+SP2-PV2)*(T-Time(iCount))/2.0
CO2=KC2*(SP2-PV2)+Kci2*IE
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)
F(6)=-U(6)

```

(4)

Case(2)

```
PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)
PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)
+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)
```

```
If(iRes==9)Ei1(iCount)=SP1-PV1
IE=0.0
Do i=1,iCount-1
  IE=IE+(Ei1(i+1)+Ei1(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(Ei1(iCount)+SP1-PV1)*(T-Time(iCount))/2.0
CO1=KC1*(SP1-PV1)+Kci1*IE
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)+KPD1*DS
F(6)=-U(6)+KPD2*DS
```

```
If(iRes==9)Ei2(iCount)=SP2-PV2
IE=0.0
Do i=1,iCount-1
  IE=IE+(Ei2(i+1)+Ei2(i))*(Time(i+1)-Time(i))/2.0
EndDo
IE=IE+(Ei2(iCount)+SP2-PV2)*(T-Time(iCount))/2.0
CO2=KC2*(SP2-PV2)+Kci2*IE
F(1)=-U(1)+KP11*CO1
F(2)=-U(2)+KP12*CO2
F(3)=-U(3)+KP21*CO1
F(4)=-U(4)+KP22*CO2
F(5)=-U(5)+KPD1*DS
F(6)=-U(6)+KPD2*DS
```

End Select

(5)

ISD,Collect History of the Process Variables and Control Output

```
If (iRes==9)Then
  PV1i(iCount)=PV1
  CO1i(iCount)=CO1
  PV2i(iCount)=PV2
  CO2i(iCount)=CO2
EndIf
```

@Coefficient Matrix

```
E(1)=Tau11
E(2)=Tau12
E(3)=Tau21
E(4)=Tau22
E(5)=TauD1
E(6)=TauD2
```

@Solver Options

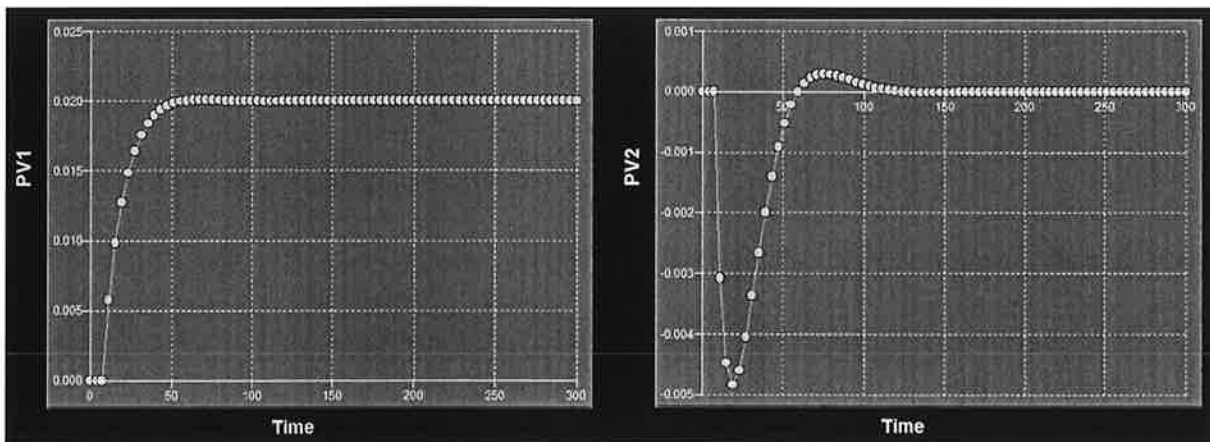
```
Neq=6
Npts=75
Tend=300.0
Tstop=7.0
```

```
Param(28)=1          ISD,Continue Integration Past Dead Time
Param(177)=1         ISD,Enable Integration History Access
VariableNames=Time;Y1;Y2;Y3;Y4;Y5;Y6    ISD,Variable Names
Append=PV1;PV2       ISD,Display Process Variable and
Control Output
```

@After Calling Solver

```
If(iCount==0)Then
  PV1=0.0
  PV2=0.0
  CO1=KC1*(SP1-PV1)
  CO2=KC2*(SP2-PV2)
Else
  PV1=QuadraticSpline(Time(1:iCount),PV1i(1:iCount),Tini)
  CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini)
  PV2=QuadraticSpline(Time(1:iCount),PV2i(1:iCount),Tini)
  CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini)
EndIf
```

(6)



Problem 2; Question 10 (Complete Q9 Using Cohen-Coon) :

!PID Controllers

!-----

!Regulatory-Control Implementation in Time-Domain

!-----

!Written by: Shereen Daik

!*****

Use AthenaUtilities

Global Tau11,Tau12,Tau21,Tau22,Tau1,Tau2 As Real

Global KC1,KC2,KC11,KC12,KCD1,KCD2,SPXD,SPXB,DS As Real

Global KP11,KP12,KP21,KP22,KP1,KP2 As Real

Global THETA11,THETA12,THETA21,THETA22,Thetap As Real

Global StabM, Ti1, Ti2,Thetad, KU1,KU2,PU1,PU2 As Real

Global Time(5000),Y1i(5000),Y2i(5000 As Real

Global Y3i(5000),Y4i(5000), Y5i(5000),Y6i(5000) As Real

Global CO1i(5000),CO2i(5000),PV1i(5000),PV2i(5000) As Real

Global E1i(5000), E2i(5000) As Real

Global iCount As Integer

Thetap=0.0 !SD,Process Dead Time

Thetad=7.0 !SD,Disturbance Dead time

Tau11=10.0 !SD,Process Lag Time

Tau21=16.0 !SD,Process Lag Time

Tau12=24.0 !SD,Process Lag Time

Tau22=22.0 !SD,Process Lag Time

TauD1=22.0 !SD,Disturbance Lag Time

TauD2=24.0 !SD,Disturbance Lag Time

KP11=0.1247 !SD,Process Gain

KP12=0.546 !SD,Process Gain

KP21=-0.1246!SD,Process Gain

KP22=1.116 !SD,Process Gain

KPD1=0.588 !SD,Disturbance Gain

KPD2=1.41 !SD,Disturbance Gain

StabM=2.0 !SD,Stability Margin

Ku1=30.53 !SD,Ultimate Gain

Ku2=6.78 !SD,Ultimate Gain

Pu1=17.10 !SD,Ultimate Period

Pu2=18.43 !SD,Ultimate Period

KC1=32.08 !SD,Proportional Gain

KC2=3.58 !SD,Proportional Gain

Kci1=3.208 !SD,Integral Gain

Kci2=0.163 !SD,Integral Gain

Kcd1=0.0 !SD,Derivative Gain

Kcd2=0.0 !SD,Derivative Gain

!SD,Cohen-Coon

!-----

If(Kci1>0.0 .AND. Kcd1==0.0)Then !SD,Proportional-Integral Controller

KC1=0.9/(StabM*KP11)*(Tau11/Thetad+0.092)

Ti1=3.33*Thetad*((Tau11+0.092*Thetad)/(Tau11+2.22*Thetad))

Kci1=KC1/Ti1

EndIf

If(Kci2>0.0 .AND. Kcd2==0.0)Then !SD,Proportional-Integral Controller

KC2=0.9/(StabM*KP22)*(Tau11/Thetad+0.092)

Ti2=3.33*Thetad*((Tau22+0.092*Thetad)/(Tau22+2.22*Thetad))

Kci2=KC2/Ti2

EndIf

(1)

iCount=0 !SD,Counter for the History Profiles

SP1=0.02 !SD,Process Variable Set Point Value of Xd

SP2=0.0 !SD,Process Variable Set Point Value of Xb

DS=0 !SD,Disturbance Set Point Value

@Initial Conditions

U(1:Neq)=0.0

@Model Equations

Dim IE As Real

Dim i As Integer

!SD,Collect the History of the Integrated Model Variables

!-----

If (iRes==9)Then

iCount=iCount+1

Time(iCount)=T

Y1i(iCount)=U(1)

Y2i(iCount)=U(2)

Y3i(iCount)=U(3)

Y4i(iCount)=U(4)

Y5i(iCount)=U(5)

Y6i(iCount)=U(6)

EndIf

!SD,Formulate the Feedback Control Loop Equations

!-----

Select Case(iRange)

Case(0)

PV1=U(1)+U(2)+U(5)

PV2=U(3)+U(4)+U(6)

If(iRes==0)Ei1(iCount)=SP1-PV1

CO1=KC1*(SP1-PV1)+Kci1*(SP1-PV1)*T

F(1)=-U(1)

F(2)=-U(2)

F(3)=-U(3)

F(4)=-U(4)

F(5)=-U(5)

F(6)=-U(6)

If(iRes==0)Ei2(iCount)=SP2-PV2

CO2=KC2*(SP2-PV2)+Kci2*(SP2-PV2)*T

F(1)=-U(1)

F(2)=-U(2)

F(3)=-U(3)

F(4)=-U(4)

F(5)=-U(5)

F(6)=-U(6)

Case(1)

PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad)

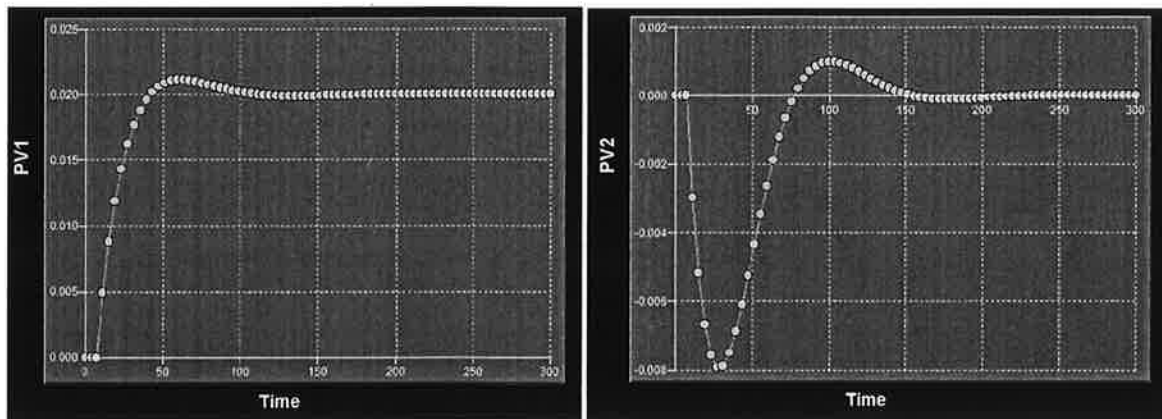
PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap)

+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad)

(2)

<pre> If(iRes==9)Ei1(iCount)=SP1-PV1 IE=0.0 Do i=1,iCount-1 IE=IE+(Ei1(i+1)+Ei1(i))*(Time(i+1)-Time(i))/2.0 EndDo IE=IE+(Ei1(iCount)+SP1-PV1)*(T-Time(iCount))/2.0 CO1=KC1*(SP1-PV1)+Kci1*IE F(1)=-U(1)+KP11*CO1 F(2)=-U(2)+KP12*CO2 F(3)=-U(3)+KP21*CO1 F(4)=-U(4)+KP22*CO2 F(5)=-U(5) F(6)=-U(6) If(iRes==9)Ei2(iCount)=SP2-PV2 IE=0.0 Do i=1,iCount-1 IE=IE+(Ei2(i+1)+Ei2(i))*(Time(i+1)-Time(i))/2.0 EndDo IE=IE+(Ei2(iCount)+SP2-PV2)*(T-Time(iCount))/2.0 CO2=KC2*(SP2-PV2)+Kci2*IE F(1)=-U(1)+KP11*CO1 F(2)=-U(2)+KP12*CO2 F(3)=-U(3)+KP21*CO1 F(4)=-U(4)+KP22*CO2 F(5)=-U(5) F(6)=-U(6) </pre> (3)	<pre> F(1)=-U(1)+KP11*CO1 F(2)=-U(2)+KP12*CO2 F(3)=-U(3)+KP21*CO1 F(4)=-U(4)+KP22*CO2 F(5)=-U(5)+KPD1*DS F(6)=-U(6)+KPD2*DS End Select !SD,Collect History of the Process Variables and Control Output ----- If (iRes==9)Then PV1i(iCount)=PV1 CO1i(iCount)=CO1 PV2i(iCount)=PV2 CO2i(iCount)=CO2 EndIf @Coefficient Matrix E(1)=Tau11 E(2)=Tau12 E(3)=Tau21 E(4)=Tau22 E(5)=TauD1 E(6)=TauD2 </pre> (5)
<pre> Case(2) PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-Thetap) +QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-Thetap) +QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-Thetad) PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-Thetap) +QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-Thetap) +QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-Thetad) If(iRes==9)Ei1(iCount)=SP1-PV1 IE=0.0 Do i=1,iCount-1 IE=IE+(Ei1(i+1)+Ei1(i))*(Time(i+1)-Time(i))/2.0 EndDo IE=IE+(Ei1(iCount)+SP1-PV1)*(T-Time(iCount))/2.0 CO1=KC1*(SP1-PV1)+Kci1*IE F(1)=-U(1)+KP11*CO1 F(2)=-U(2)+KP12*CO2 F(3)=-U(3)+KP21*CO1 F(4)=-U(4)+KP22*CO2 F(5)=-U(5)+KPD1*DS F(6)=-U(6)+KPD2*DS If(iRes==9)Ei2(iCount)=SP2-PV2 IE=0.0 Do i=1,iCount-1 IE=IE+(Ei2(i+1)+Ei2(i))*(Time(i+1)-Time(i))/2.0 EndDo IE=IE+(Ei2(iCount)+SP2-PV2)*(T-Time(iCount))/2.0 CO2=KC2*(SP2-PV2)+Kci2*IE </pre> (4)	<pre> @Solver Options Neq=6 Npts=75 Tend=300.0 Tstop=7.0 Param(28)=1 Param(177)=1 VariableNames=Time;Y1;Y2;Y3;Y4;Y5;Y6 Append=PV1;PV2 ISD,Continue Integration Past Dead Time ISD,Enable Integration History Access ISD,Variable Names ISD,Display Process Variable and Control Output @After Calling Solver If(iCount==0)Then PV1=0.0 PV2=0.0 CO1=KC1*(SP1-PV1) CO2=KC2*(SP2-PV2) Else PV1=QuadraticSpline(Time(1:iCount),PV1i(1:iCount),Tini) CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini) PV2=QuadraticSpline(Time(1:iCount),PV2i(1:iCount),Tini) CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini) EndIf </pre> (6)



Conclusion:

Once again, based on the graphs generated by the three figures it appears that the Ziegler-Nichols tuning method is the best among the three because it has less oscillations.

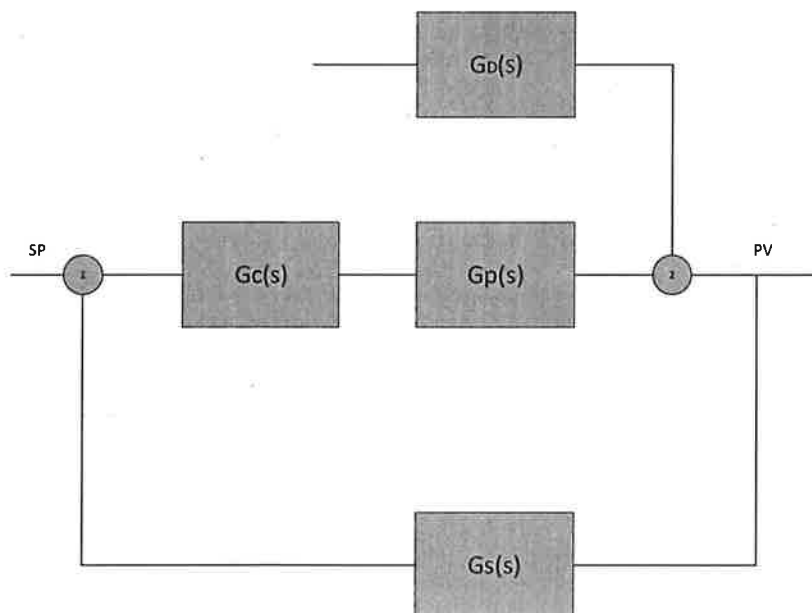
Proof for Problem 2; Question 10:

For part 10, in order to reach the equation of:

$$G_{pij} = G_{pij} e^{-s(\text{Analyzer Dead Time})}$$

we must analyze a simple control block diagram which has a transportation delay. There are two way to develop the simple control block diagram:

- 1) Allow the disturbance to act on the controller variable.



The closed loop transfer function for this block diagram would be:

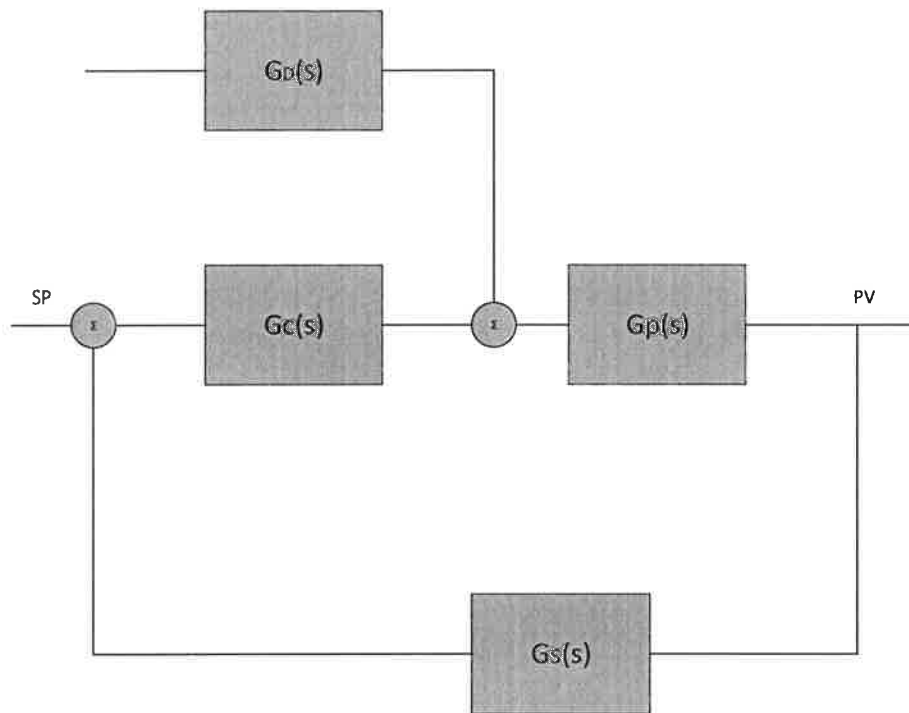
$$PV = DG_d + G_c G_p (SP - PV G_s)$$

Which simplifies to:

$$PV = \frac{G_d}{1 + G_s G_c G_p} D + \frac{G_c G_p}{1 + G_s G_c G_p} SP$$

This equation does not lead to the equation for G_{PII} listed above.

2) Allow the disturbance to act on the controller output.



In this case, the closed loop transfer function would be:

$$PV = G_p (DG_d + G_c (SP - PV G_s))$$

This simplifies to:

$$PV = \frac{G_d G_p}{1 + G_s G_c G_p} D + \frac{G_c G_p}{1 + G_s G_c G_p} SP$$

Similarly to the case in which the disturbance acting on the process variable, the disturbance acting on the controller output will not resemble the equation for G_{PIJ} listed above. The only conclusion that I can make at this moment is that the proof cannot be provided. The only scenario in which I would be able to prove the equation listed above would be if I set $SP=0$. In this case then I can prove the equation for G_{PIJ} .

